Graph Reduction with Graph Neural Networks for Column Generation in Time-Dependent Vehicle Routing

SNU SCM

Sejeong Kang

2026. 02. 25.

1. Introduction

Time-Dependency in Urban Logistics

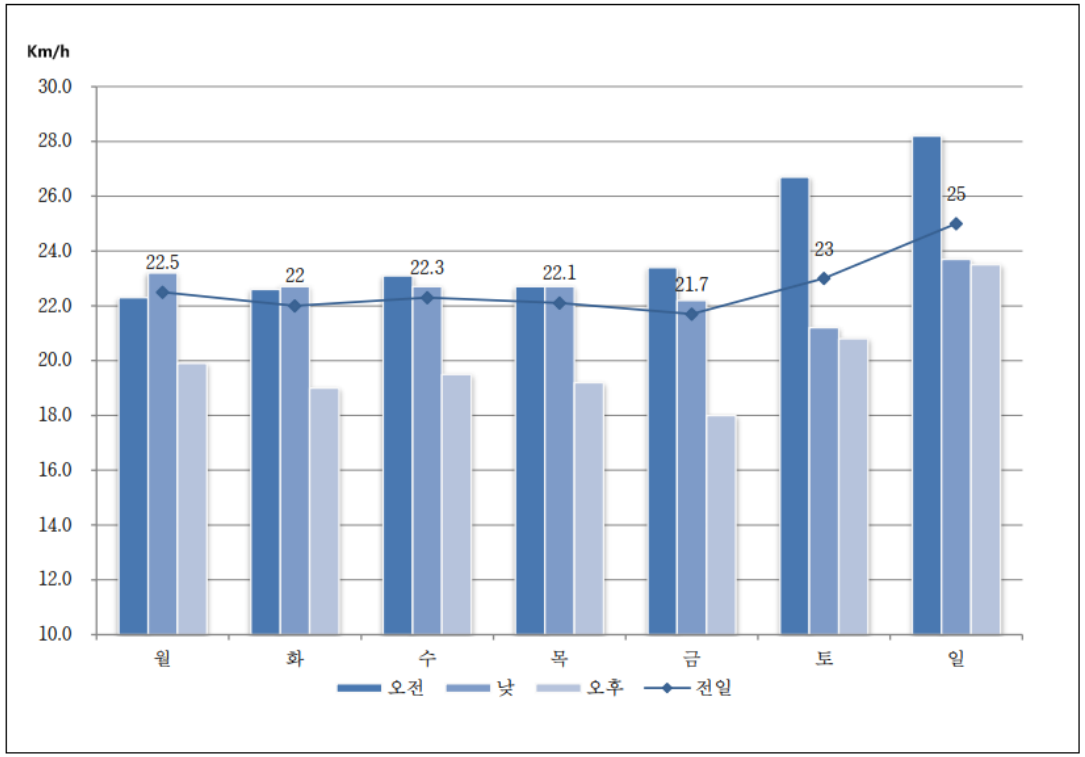
- Continuous Time Dependency: Traffic Congestion
  - ✓ Time-Dependent VRPs (TDVRPs) primarily model traffic congestion by representing travel times as continuous functions of departure time

- Modeling in TDVRP Literature
  - ✓ Most Time-Dependent VRP models represent congestion using continuous, piecewise-defined travel time functions

- Key Characteristic
  - ✓ Small changes in departure time lead to small changes in travel time

(단위:km/h)

구분	시간대	통행속도									2023년		2023년 대비	
		월	화	수	목	금	토	일	평일	주말	평일	주말	평일	주말
전일	06:00-22:00	22.5	22.0	22.3	22.1	21.7	23.0	25.0	22.1	24.0	22.2	24.2	-0.1	-0.2
오전	07:00-09:00	22.3	22.6	23.1	22.7	23.4	26.7	28.2	22.8	27.5	22.9	27.7	-0.1	-0.2
낮	12:00-14:00	23.2	22.7	22.7	22.7	22.2	21.2	23.7	22.7	22.4	22.7	22.4	0.0	0.0
오후	17:00-19:00	19.9	19.0	19.5	19.2	18.0	20.8	23.5	19.1	22.2	19.0	22.3	0.1	-0.1



Source: 2024 Seoul Vehicle Traffic Speed Report (Seoul Metropolitan Government)
https://topis.seoul.go.kr/refRoom/openRefRoom_1.do

1. Introduction

Urban Access Restrictions

- Time-Dependent Access Regulations
  - ✓ Examples: Limited Traffic Zones (LTZ), Low Emission Zones (LEZ)
  - ✓ Certain roads are **fully closed** to freight vehicles during **specific time windows**, creating discontinuous travel time functions
- Feasibility and Trade-Offs
  - ✓ Access restrictions directly affect route feasibility and may force vehicles to either wait or detour
- Modeling Implications
  - ✓ These regulations necessitate time-aware and path-flexible routing models capable of handling discontinuities

Country	Permissible total weight	When?	Special characteristics
Austria	from 7.5 t (semi-trailer trucks from 3,5 t)	Sundays and public holidays from 0 am to 10 pm; Saturdays from 3 pm to 10 pm; at night from 10 pm to 5 am	ban on night driving
Croatia	from 7.5 t	public holidays from 2 pm to 11 pm, on days before a public holiday from 3 pm to 11 pm	additional driving bans during the vacations (see below)
Czech Republic	from 7.5 t	Sundays and public holidays from 1 pm to 10 pm	additional driving bans during the vacations (see below)
France	from 7.5 t	Sundays and public holidays from 0 am to 10 pm; Saturdays and the day before multi-day public holidays from 10 pm to midnight	additional driving bans during the vacations (see below)

Source : DHL

1. Introduction

Urban Access Restrictions

- Time-Dependent Access Regulations
  - ✓ Examples: Limited Traffic Zones (LTZ), Low Emission Zones (LEZ)
  - ✓ Certain roads are **fully closed** to freight vehicles during **specific time windows**, creating discontinuous travel time functions
- Feasibility and Trade-Offs
  - ✓ Access restrictions directly affect route feasibility and may force vehicles to either wait or detour
- Modeling Implications
  - ✓ These regulations necessitate time-aware and path-flexible routing models capable of handling discontinuities

Source : https://truckban.eu/Germany

1. Introduction

Time Dependent Travel Time VRP

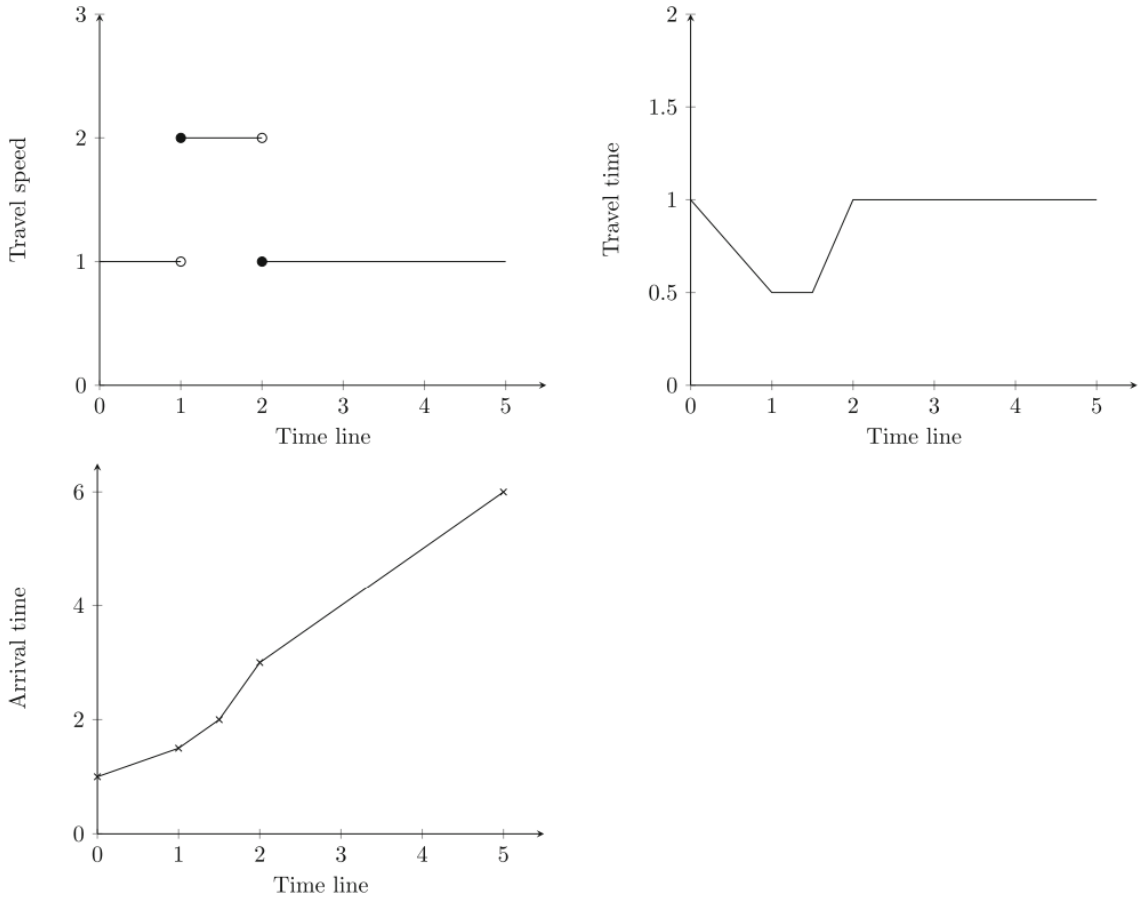
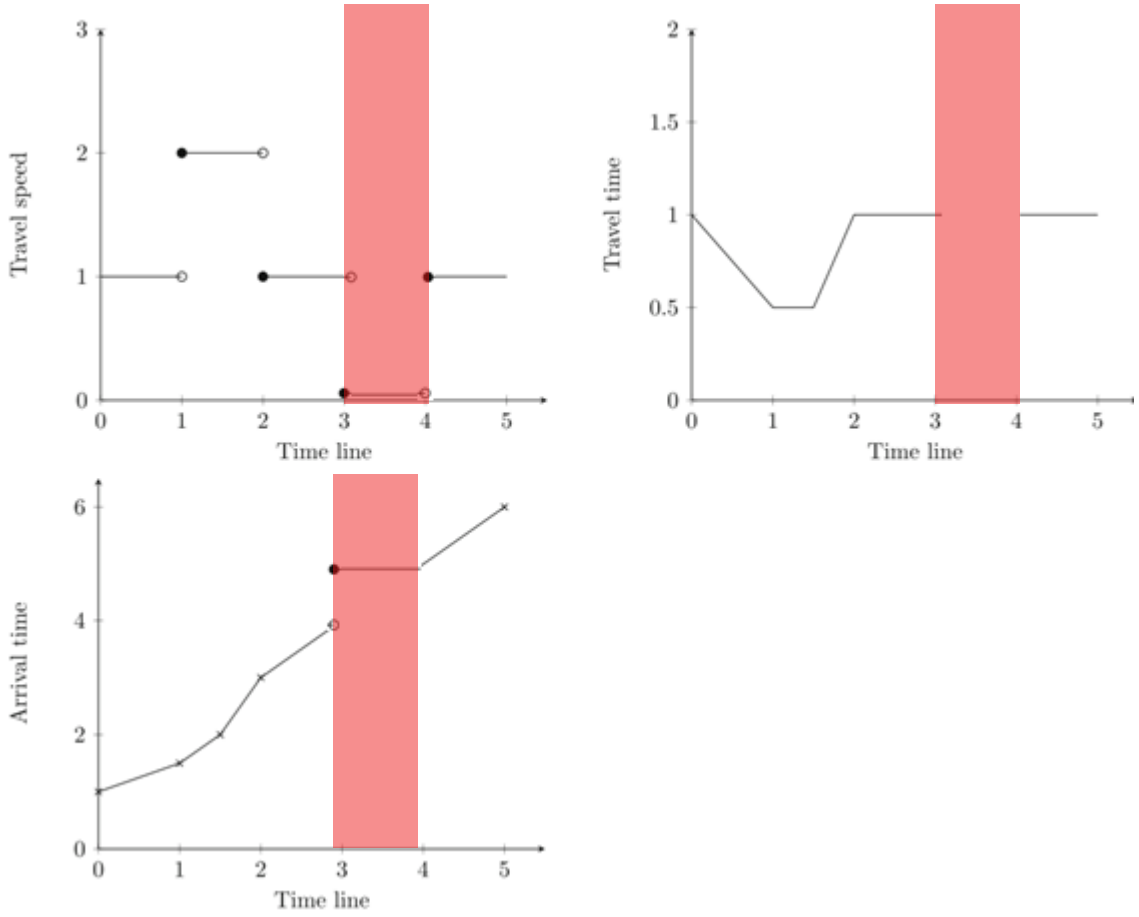


Fig. 2 Speed, travel and arrival times functions for a given arc, with three time zones on interval [0,5]

Ben Ticha, H., Absi, N., Feillet, D. et al. The Time-Dependent Vehicle Routing Problem with Time Windows and Road-Network Information. SN Oper. Res. Forum 2, 4 (2021).

Composite Time Dependent Travel Time VRP



2. Literature Review

Comparison of Multigraph Representations and Pricing Complexity across VRP Variants

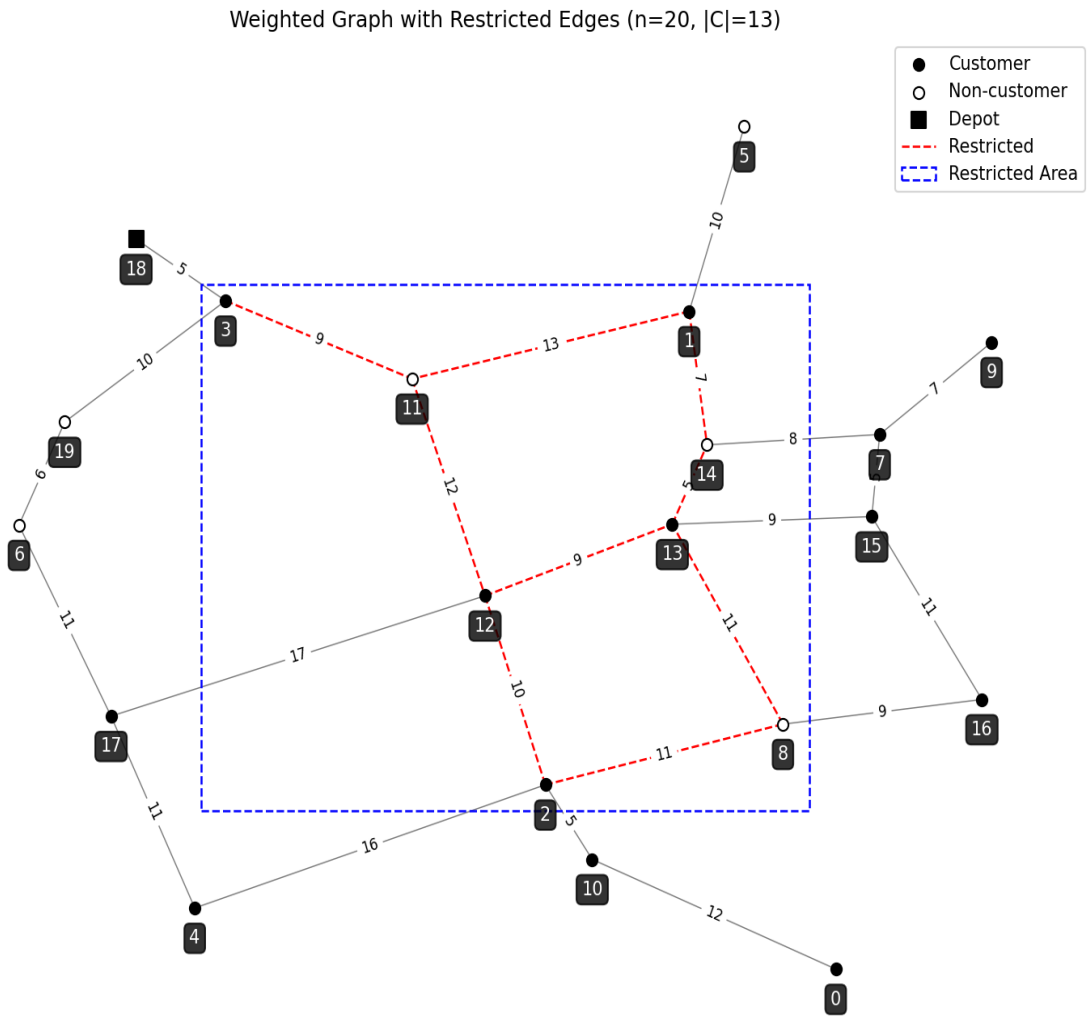
VRP Variant	Graph Representation	Arrival Function	Path Composition Complexity	Validity of Dominance Rules
CVRP	Customer graph	Constant (scalar)	N/A	Valid
TDVRP (Continuous)	Customer graph	Continuous PLF	Polynomial	Valid (via PLF comparison)
VRPAR (Discontinuous)	Multigraph	Discontinuous PLF	High	Valid
C-TD-VRP (This study)	**Multigraph**	**Composite discontinuous PLF**	**Higher**	**Invalidated**

Munise Kübra Şahin, Hande Yaman (2024) The Vehicle Routing Problem with Access Restrictions. Transportation Science 58(5):1101-1120.
https://doi.org/10.1287/trsc.2023.0261

3. Problem Definition

Multigraph Model for VRPSPD-AR

- Nodes
  - ✓ Inbound Cross Dock (single)
  - ✓ Suppliers
  - ✓ Non-customer intersections
- Arcs
  - ✓ actual road segments
- Why not customer-based VRP?
  - ✓ Traditional models connect customers directly → unrealistic under urban restrictions.
  - ✓ Intersection nodes let us capture legal paths, detours, and waiting precisely.

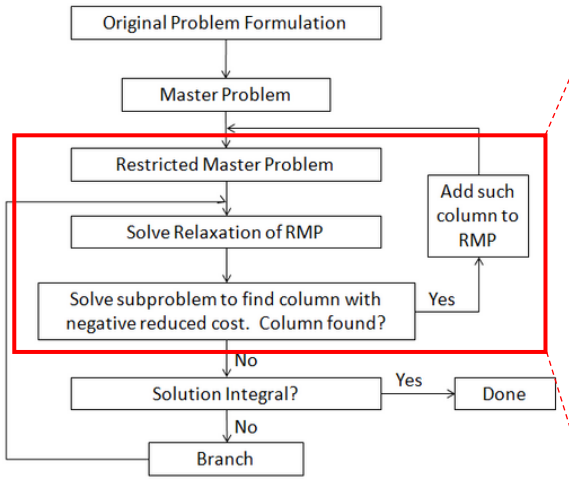


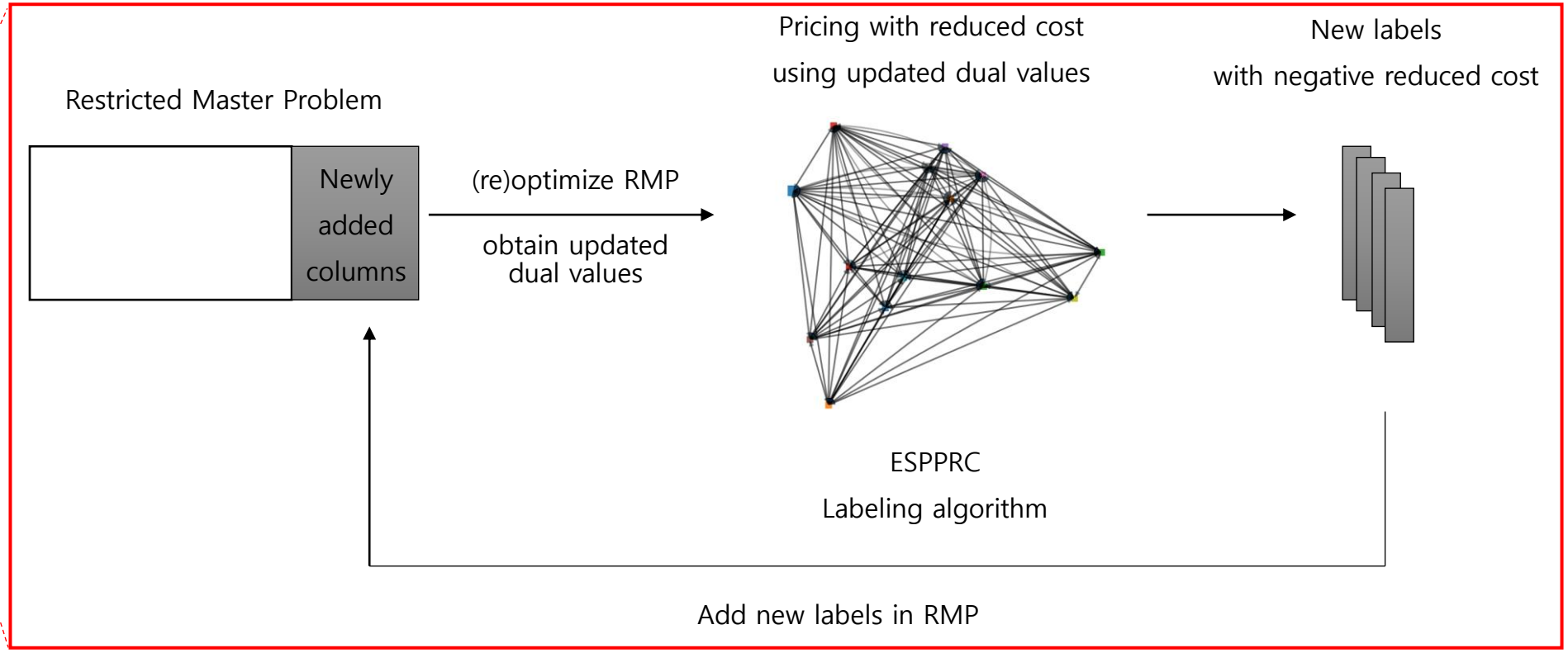
Road network (arc based) representation

3. Problem Definition

Methodology Preview: Column Generation Framework

- Solving a Path/Label Selection Problem via an RMP–Pricing Loop
  - ✓ Generate improving labels/paths and refine the master solution iteratively





3. Problem Definition

Master Problem

- Path based Set Covering Problem
  - ✓ Master Problem selects the optimal set of routes to cover all customers at minimum cost

$$
\begin{aligned}
\text{(MP)} \quad \min_{z} \quad & \sum_{w \in W} (c_w + f)\, z_w \\
\text{s.t.} \quad & \sum_{w \in W} a_{jw}\, z_w \geq 1, \quad \forall j \in J, \\
& z_w \in \{0,1\}, \qquad \forall w \in W.
\end{aligned}
$$

- Implicit Definition of Feasible Routes
  - ✓ Capacity Constraint: The total demand does not exceed vehicle capacity

$$\sum_{j \subset J} a_{jw} d_j \leq Q, \quad \forall w \in W.$$

  - ✓ Time Horizon Constraint: The vehicle returns to the depot before time T

▷ Time-Dependent Travel Time Model

$$A_w(0) \leq T, \quad \forall w \in W.$$

3. Problem Definition

Restricted Master Problem

- Primal Restricted Master Problem
  - ✓ Since the cardinality of W is exponentially large, we solve the LP relaxation over a subset $\bar{W}$

$$
\begin{aligned}
\text{(RMP)} \quad \min_{z,\,u} \quad & \sum_{w \in \bar{W}} (c_w + f)\, z_w \;+\; M \sum_{j \in J} u_j \\
\text{s.t.} \quad & \sum_{w \in \bar{W}} a_{jw}\, z_w + u_j \;\geq\; 1, \qquad \forall j \in J, \quad [\alpha_j] \\
& z_w \geq 0, \quad \forall w \in \bar{W}, \quad u_j \geq 0, \quad \forall j \in J.
\end{aligned}
$$

3. Problem Definition

Restricted Master Problem

- Dual Restricted Master Problem

$$
\begin{aligned}
\text{(Dual)} \quad \max_{\alpha} \quad & \sum_{j \in J} \alpha_j \\
\text{s.t.} \quad & \sum_{j \in J} a_{jw}\, \alpha_j \;\leq\; c_w + f, \quad \forall w \in \bar{W}, \\
& \alpha_j \leq M, \qquad\qquad \forall j \in J.
\end{aligned}
$$

- Reduced cost of a candidate route
  - ✓ The pricing problem seeks a route with $\overline{C_w} < 0$

$$
\bar{c}_w \;=\; (c_w + f) \;-\; \sum_{j \in J} a_{jw}\, \alpha_j
$$

3. Problem Definition

Pricing Problem (Column Generation)

- Elementary Shortest Path Problem with Resource Constraints (ESPPRC) ▶ Labeling Algorithm
  - ✓ Pricing problem is modeled as an ESPPRC on a multigraph

$$
\begin{aligned}
\min_{x} \quad & f + \sum_{(i,j)\in V'\times V'} \sum_{e\in\mathcal{P}_{ij}} (c_e - \alpha_j) x_e \\
\text{s.t.} \quad & \text{Flow conservation constraints at all nodes,} \\
& \sum_{e} d_{\text{head}(e)} x_e \le Q, \quad \text{(Capacity)} \\
& \text{Time feasibility with } \tau_e(t), \quad \text{(Time Horizon)} \\
& \text{Elementarity (no cycles).}
\end{aligned}
$$

- Multigraph Construction ▶ Multigraph Construction Detail
  - ✓ Each edge (pareto-optimal path) has cost, travel time function, efficient periods, reduced cost contribution

3. Problem Definition

Branching Strategy: Successor Branching

- Select one from pairs where X_{ij} is fractional and close to 0.5

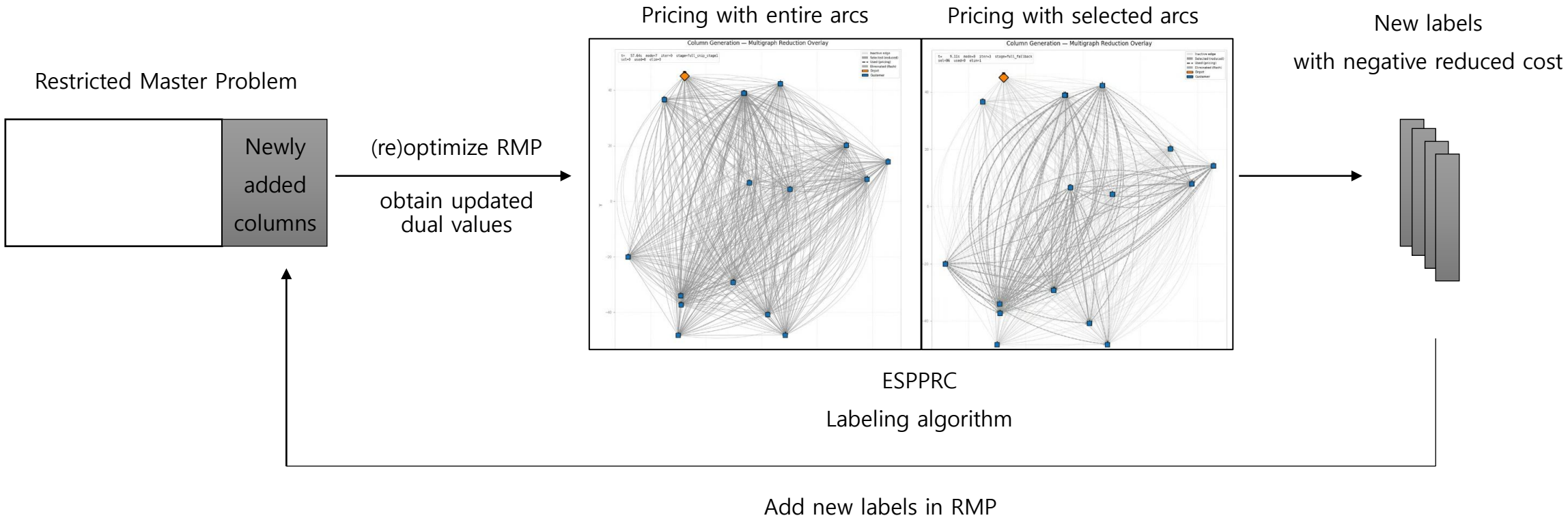
$$X_{ij} = \sum_{w \in \bar{W}} \delta(i,j)_w \, z_w^*$$

- Branching Constraints
  - ✓ Enforce Branch (i → j): The vehicle must visit j immediately after i
  - ✓ Forbid Branch (i ↛ j): The vehicle cannot visit j immediately after i

4. Solving TD-CVRP

Reduce-then-Optimize Framework

- ✓ Arcs contained in the true optimal solution (or the LP-relaxed optimal solution) tend to be very sparse
- ✓ The key bottleneck of the pricing problem (ESPPRC) is that it still considers many non-promising arcs



4. Solving TD-CVRP

Proposed Framework

- Why rebuild the network every iteration?
  - ✓ The "promisingness" of arcs is not static; it changes with current duals and branching constraints in the RMP.
  - ✓ The value of arcs for satisfying customer coverage evolves during column generation.

- Core idea
  - ✓ At each pricing iteration, rebuild a reduced graph Gr from the base network using the current duals and an ML selector.
  - ✓ Use an adaptive threshold / top-k rule; tune it online based on how many improving columns are found.

Algorithm 1 ML–CG with Per-Iteration Reduced Network

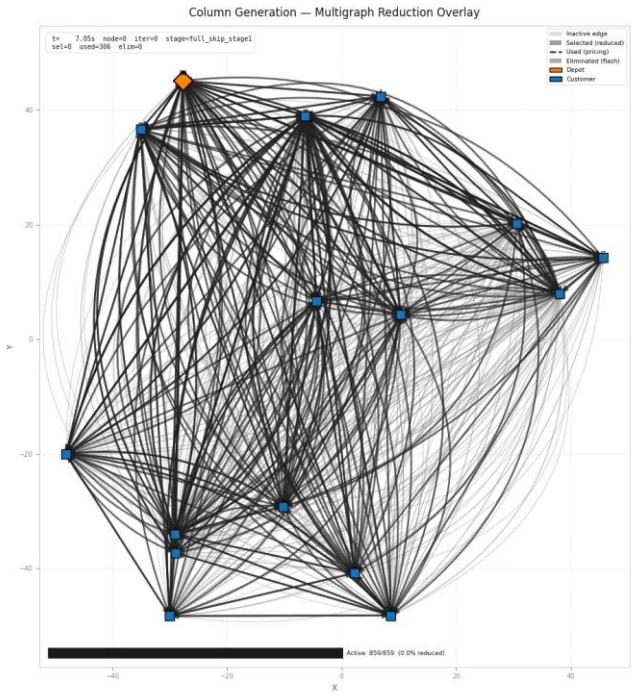
Input: base network G, depot/customers, (Q, T), RF (opt.), switch thresholds $\eta_{\min}, \eta_{\max}$
Precompute once: static arc features & parallel ranks
Initialize adaptive threshold τ
while there is an open B&B node **do**
  Solve initial RMP to get duals π
  repeat ▷ Column Generation loop
    (Reduced graph step) If ML enabled: rebuild G_r from G using current duals and τ; set $G_{\text{active}} = G_r$; else $G_{\text{active}} = G$
    Pricing: generate improving columns on G_{active} under branching constraints
    Switch rule:
    - if few columns ($< \eta_{\min}$) and using reduced $\Rightarrow$ switch to FULL; decrease τ
    - if many columns ($\geq \eta_{\max}$) and using full $\Rightarrow$ switch to REDUCED; increase τ
    - if zero columns in FULL $\Rightarrow$ stop CG at this node (optimal)

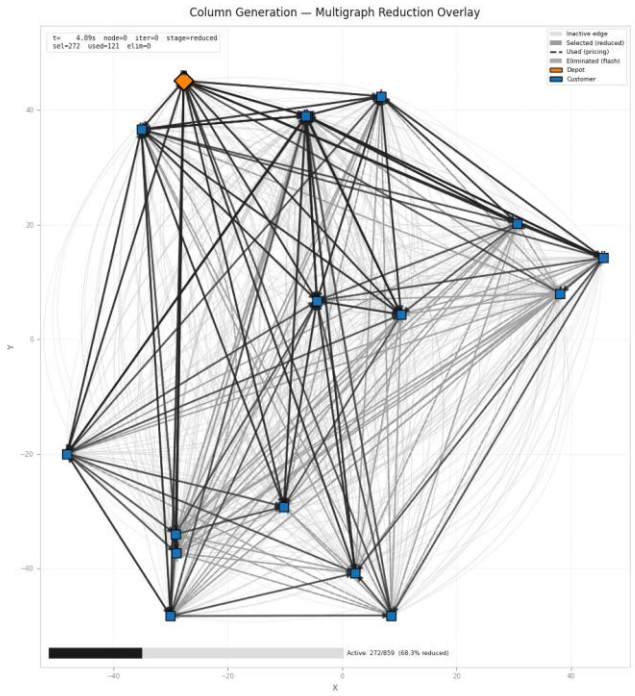
    If columns found: add to RMP, reoptimize, gently nudge τ toward target keep
  until CG stops at this node
  if RMP solution is integer **then** update incumbent and continue
  elseBranch: pick a top-k fractional arc and create two children (enforce / forbid)
  end if
end while

4. Solving TD-CVRP

Graph Reduction Strategies

- Heuristic Strategies
  - ✓ No Reduction
  - ✓ Shortest Distance
  - ✓ Minimum Time





4. Solving TD-CVRP

Graph Reduction Strategies

- GNN-based Strategies

✓ GNN Snapshot

- Run inference only once
- Reuse the same reduced graph

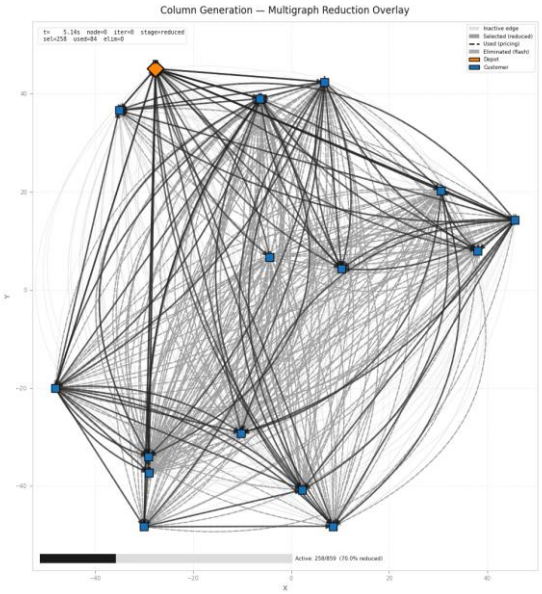
✓ GNN Reduction (Fixed τ)

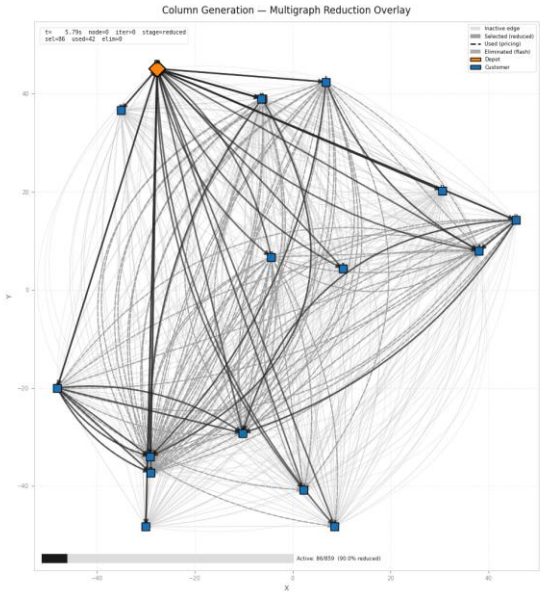
- Select only the top τ fraction of paths
- Using a fixed retention ratio τ

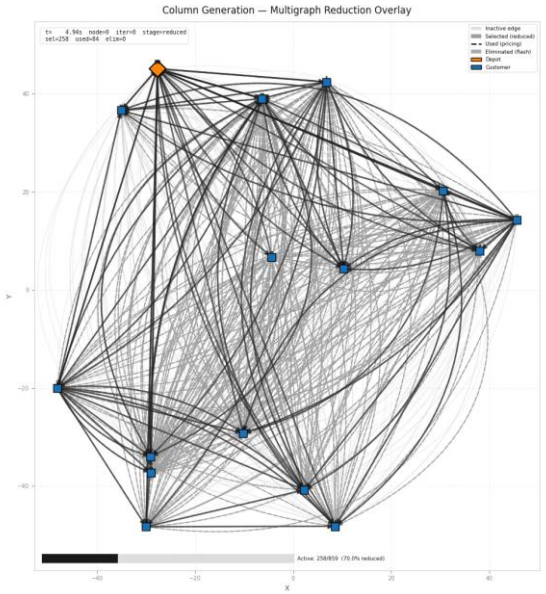
✓ GNN Reduction (Dynamic τ)

- Select only the top τ fraction of paths
- Adjust τ dynamically at each CG iteration

Adjustment logic







4. Solving TD-CVRP

Graph Reduction Strategies

- Depot Connectivity Guarantee

  ✓ After reduction, if no depot-connected path remains: restore the highest-scoring path to guarantee feasibility

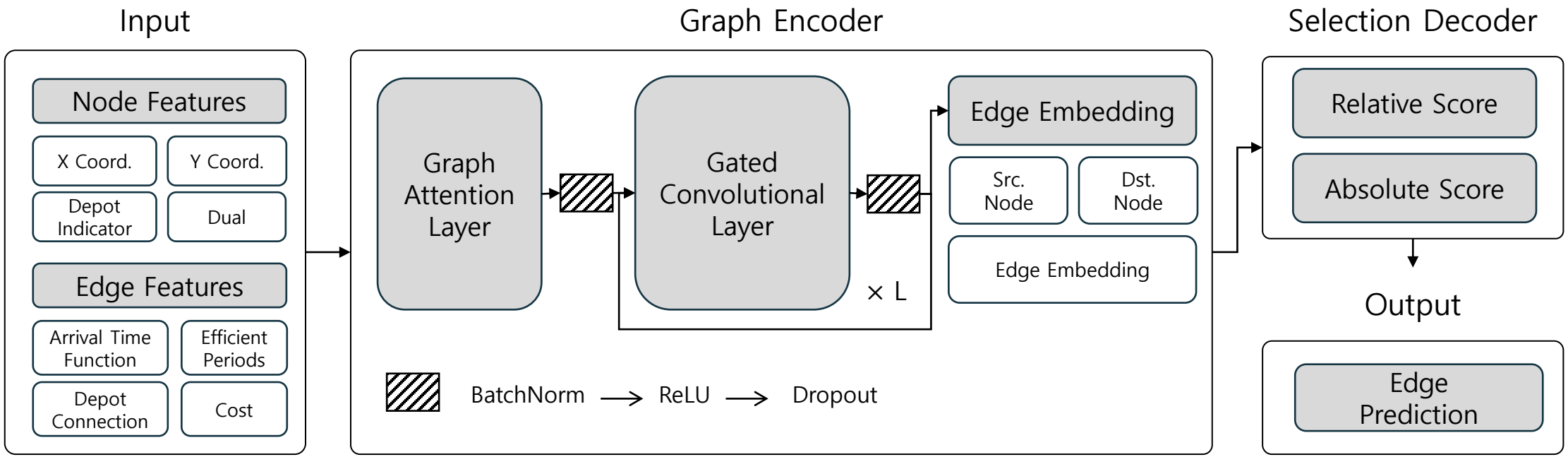
- Summary of Strategies

Strategy Name	Selection Criteria	Re-Inference Frequency
NoReduction	Keep all paths	N/A
ShortestDistance	$\arg\min(\text{Cost})$	Static (Once)
MinimumTime	$\arg\min(\text{Avg. Time})$	Static (Once)
GNN (Fixed τ)	Score $\geq$ Top $\tau\%$	Every CG Iteration
GNN (Dynamic τ)	Score $\geq$ Top $\tau_{dyn}\%$	Every CG Iteration
GNN (Snapshot)	Score $\geq$ Top $\tau\%$	Once (First Iteration only)

4. Solving TD-CVRP

GNN Architecture for Edge Classification in Multigraph

Architecture Detail

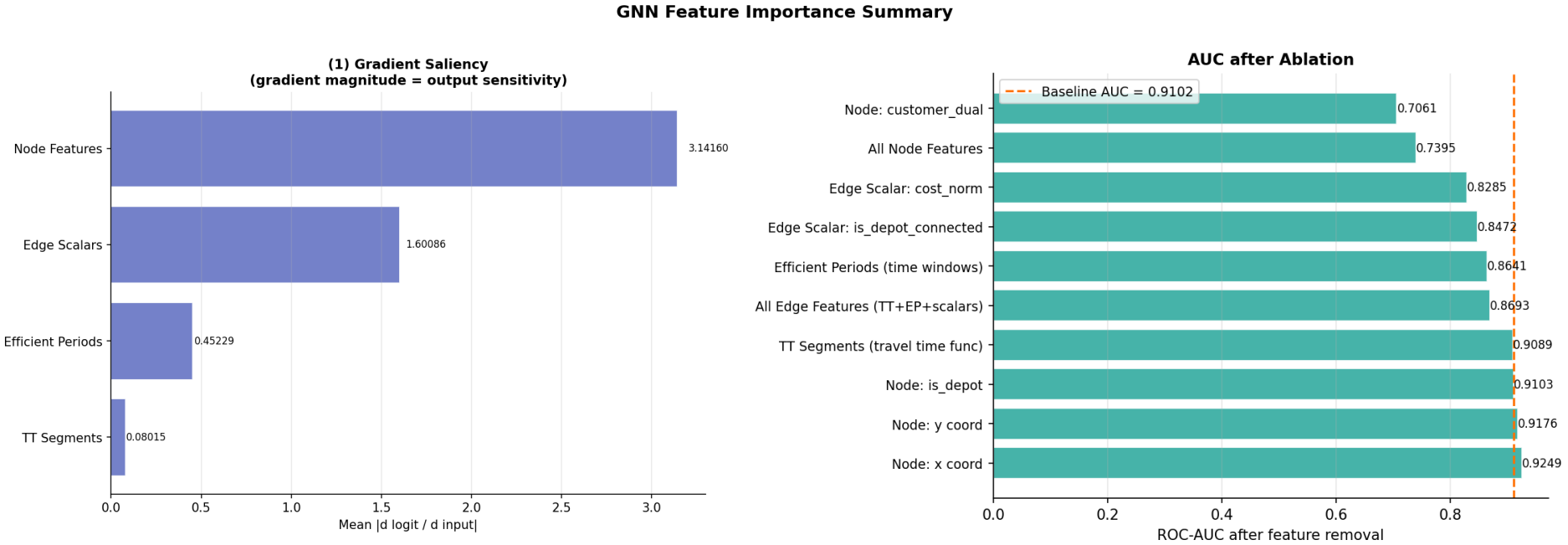


4. Solving TD-CVRP

Neural Network Training

- Goal
  - ✓ Learn a binary classifier to keep/drop arcs for a reduced network rebuilt at every CG iteration.
- Data & Split
  - ✓ Synthetic Data Generation: Refer NEWLET instances.
  - ✓ Label: y = 1 if the arc ever appeared in final routes; else 0.
  - ✓ Split (by instance): Train 400 / Val 50 / Test 50.
- Hyperparameter Tuning
  - ✓ Tuned by optuna library

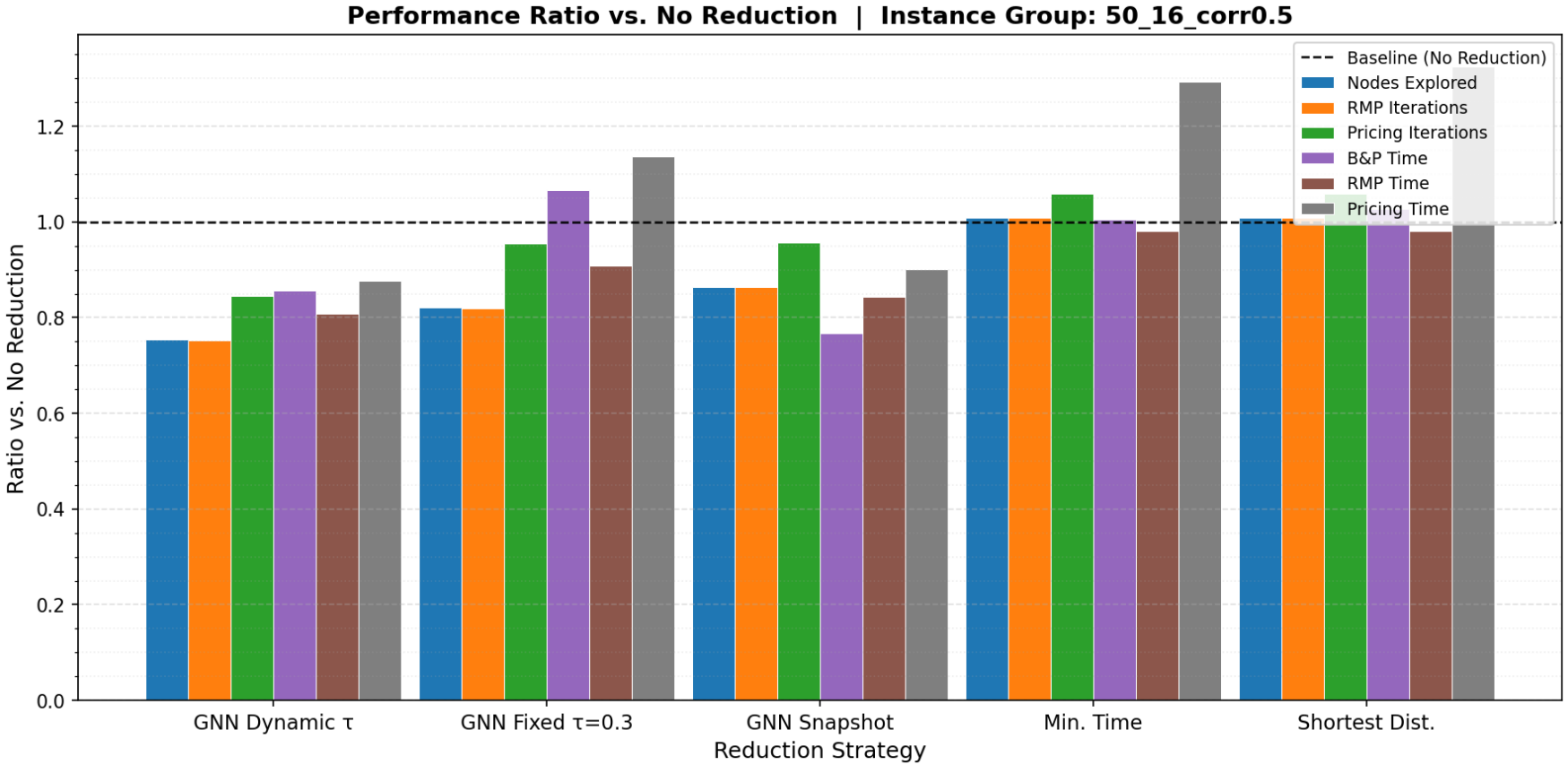
    Tuning Detail
- Feature Importance



5. Computational Experiments

Results — Baseline vs. Reduction Strategies

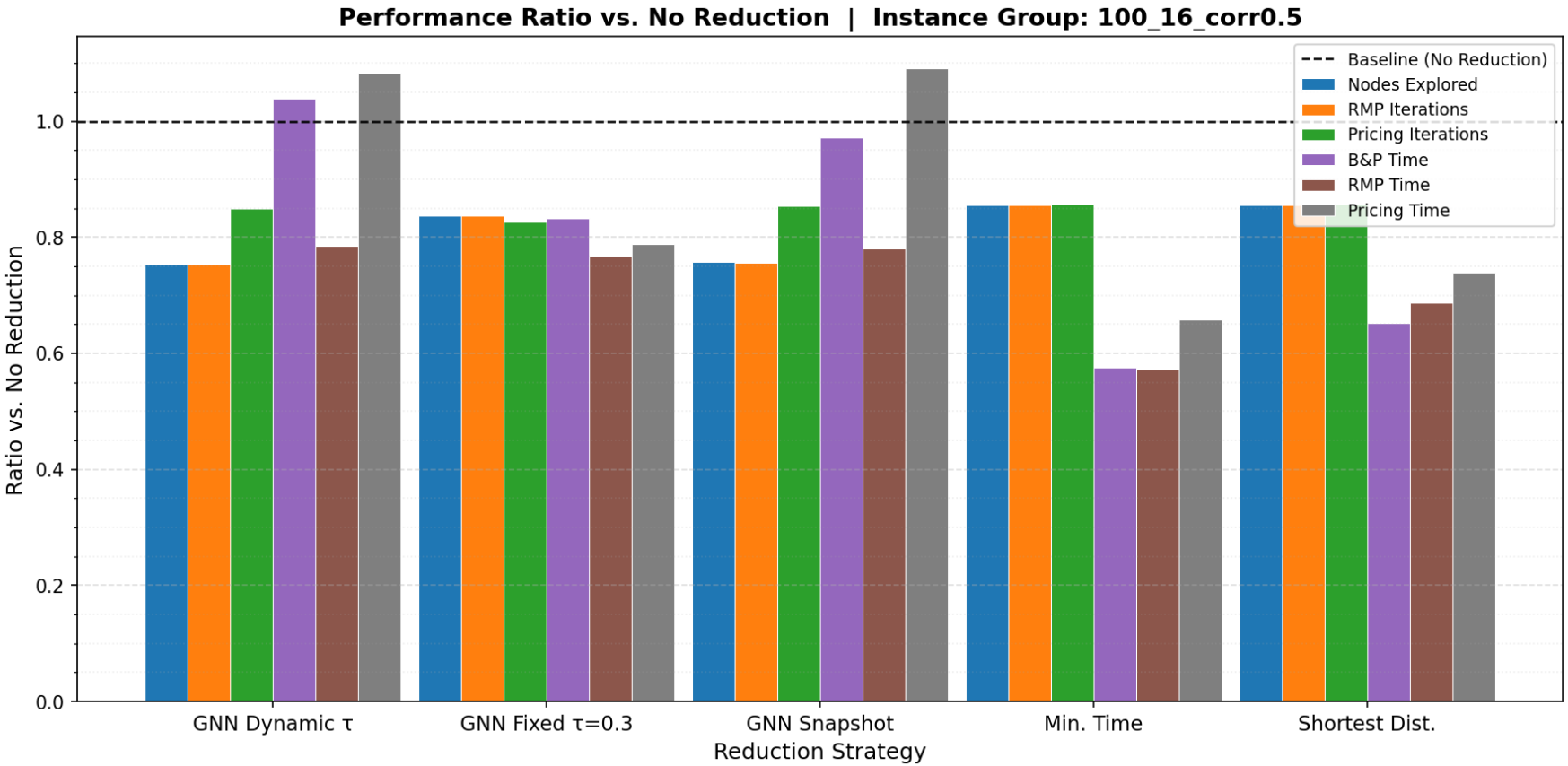
- 50-16 (Out-of-Distribution, 10 instances)
  - ✓ With No Reduction normalized to 1.0, this plot shows the mean ratio across 10 instances for each reduction strategy.
  - ✓ On average, both iterations and runtime decrease, indicating reduced computational effort.



5. Computational Experiments

Results — Baseline vs. Reduction Strategies

- 100-16 (Out-of-Distribution, 10 instances)
  - ✓ Experimental results on larger-scale settings with more road-network nodes and customers, assessing generalization performance

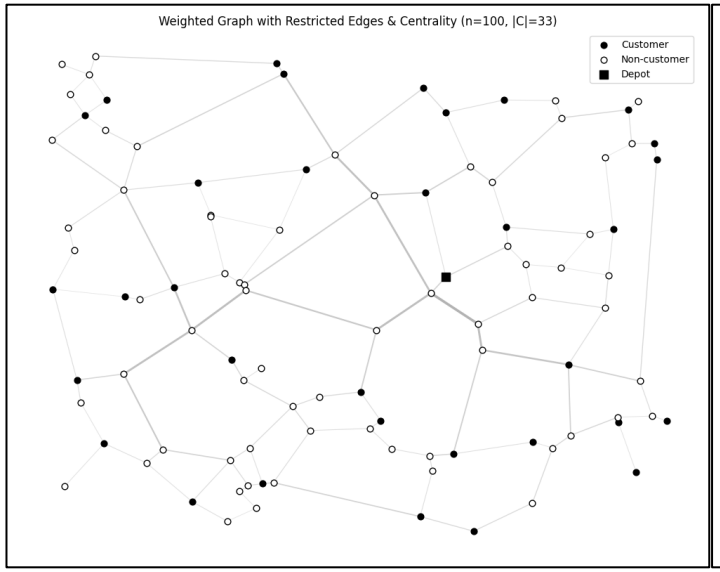


6. Limitations and Future Works

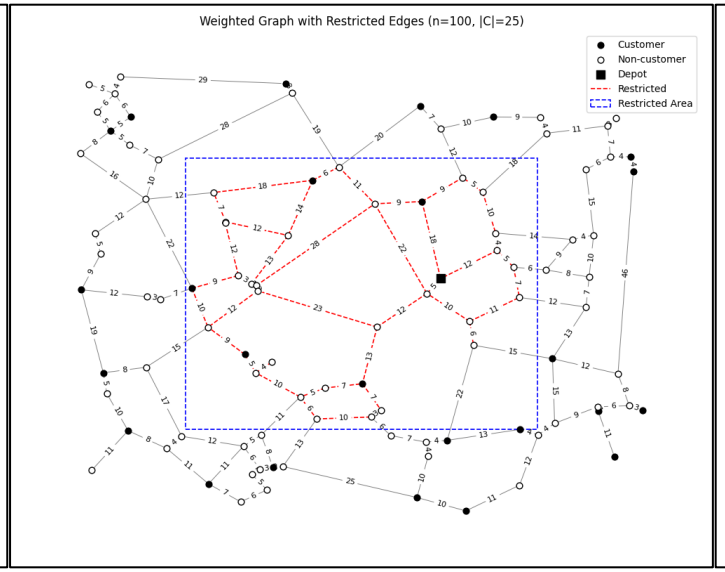
Implementation of Experiments

- Conduct experiments on instances where the number of iterations or total computation time shows significant differences in performance

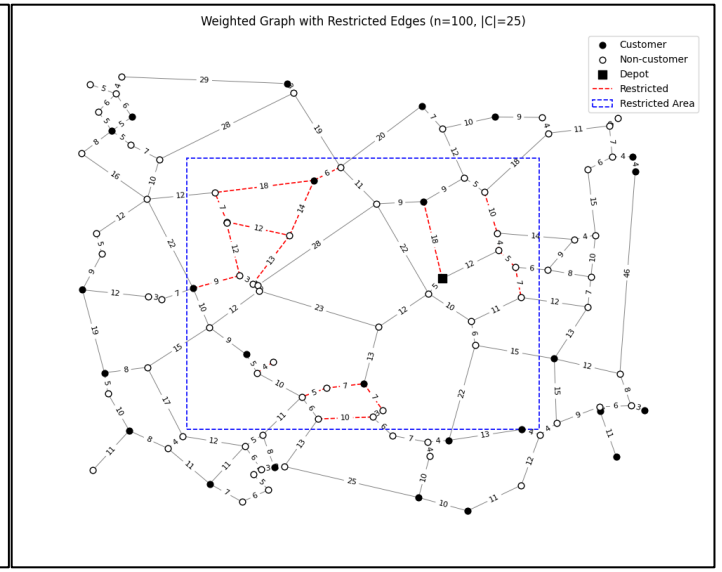
✓ Customer/Arc density
✓ Time horizon
✓ Customer demand
✓ Time sensitivity of speed
✓ Restricted road conditions



Edge betweenness centrality values



Roads restricted except for top **10%** highest-centrality edges

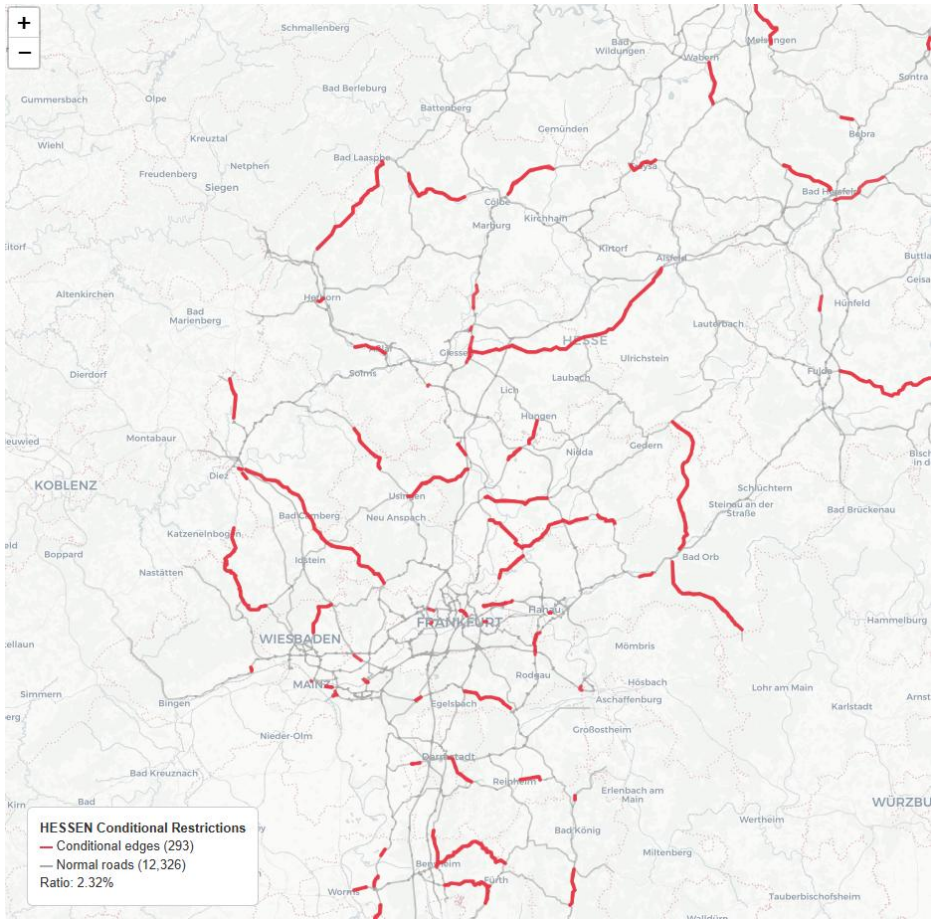


Roads restricted except for top **30%** highest-centrality edges

6. Limitations and Future Works

Implementation of Experiments

- Conduct experiments on instances from Original NEWLET / OpenStreetMap



End of Document

Appendix. Notation and Definitions (1/2)

- Sets and Indices

Symbol	Description
$G = (N, A)$	Road network graph (physical layer)
N	Set of all nodes in the road network
A	Set of all arcs (directed edges) in the road network
J	Set of customers, $J \subseteq N$
$\{0\}$	Depot node (source and sink)
W	Set of all feasible routes (columns)
$\mathcal{P}_{ij}$	Set of Pareto-optimal paths from node i to node j in the multigraph
$\bar{A} \subseteq A$	Set of restricted arcs with forbidden departure intervals

- Parameters

Symbol	Description
d_j	Demand of customer $j \in J$
Q	Vehicle capacity
T	Planning horizon (maximum route duration)
f	Fixed cost per route (vehicle dispatch cost)
δ_a	Distance of arc $a \in A$
$v_{a,k}$	Speed on arc a during time period k (time-dependent)
$[e_a, l_a)$	Restricted departure interval for arc $a \in \bar{A}$; a vehicle arriving at tail(a) in $[e_a, l_a)$ must wait until l_a
$\tau_a(t)$	Travel time on arc a when departing at time t
c_w	Travel cost (total distance) of route $w \in W$
a_{jw}	Binary parameter: $= 1$ if route w serves customer j, 0 otherwise
$\delta(i,j)_w$	Binary parameter: $= 1$ if route w visits j immediately after i

Appendix. Notation and Definitions (2/2)

- Decision Variables

Symbol	Description
z_w	Selection variable for route $w \in W$.
	$z_w \in [0,1]$ in LP relaxation, $z_w \in \{0,1\}$ in IP.
α_j	Dual variable associated with the coverage constraint for customer $j \in J$ ($\alpha_j \geq 0$)
u_j	Artificial variable for customer j (used in Big-M Phase I) ($u_j \geq 0$)

Appendix. Time-Dependent Travel Time Model

- Effective Departure Time
  - ✓ Mandatory waiting at restricted arcs

$$\hat{t}(t) = \begin{cases} l_a & \text{if } a \in \bar{A} \text{ and } e_a \leq t < l_a, \\ t & \text{otherwise.} \end{cases}$$

- Total Travel Time
  - ✓ Sum of the waiting time and the pure traversal time

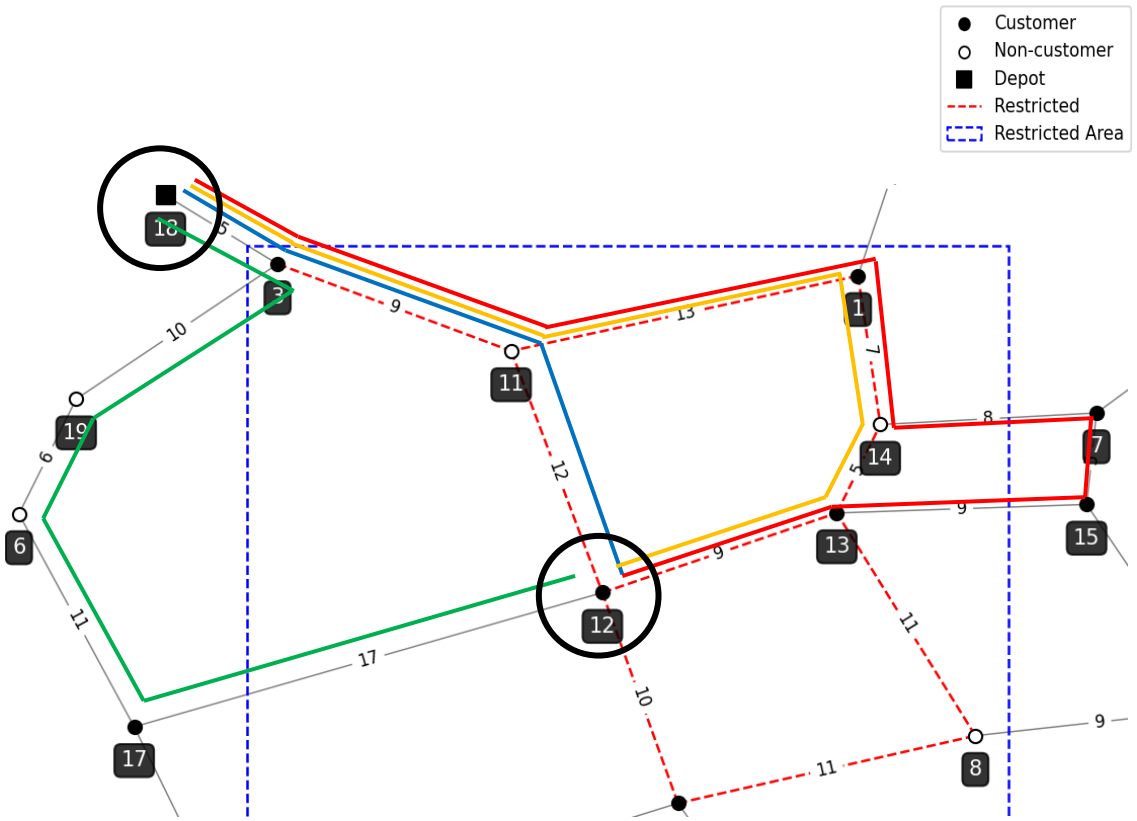
$$\tau_a(t) = \underbrace{(\hat{t}(t) - t)}_{\text{Waiting Time}} + \tilde{\tau}_a(\hat{t}(t))$$

- Pure Traversal Time
  - ✓ Integrate the speed function over period boundaries

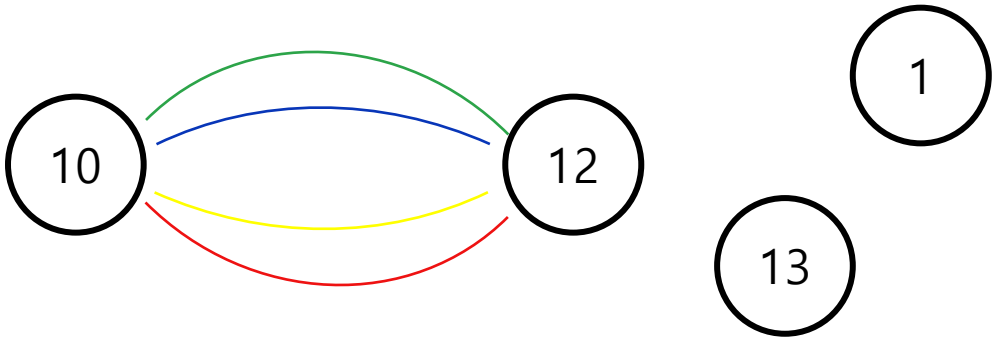
$$\tilde{\tau}_a(s) = \sum_{k=k_0}^{k^*-1} \frac{\min\left(\delta_a^{(k)},\ v_{a,k}(t_{k+1} - \max(s, t_k))\right)}{v_{a,k}} + \frac{\delta_a^{(k^*)}}{v_{a,k^*}}$$

Appendix. Multigraph Construction

Road network (arc based) representation



Multigraph (customer based) representation

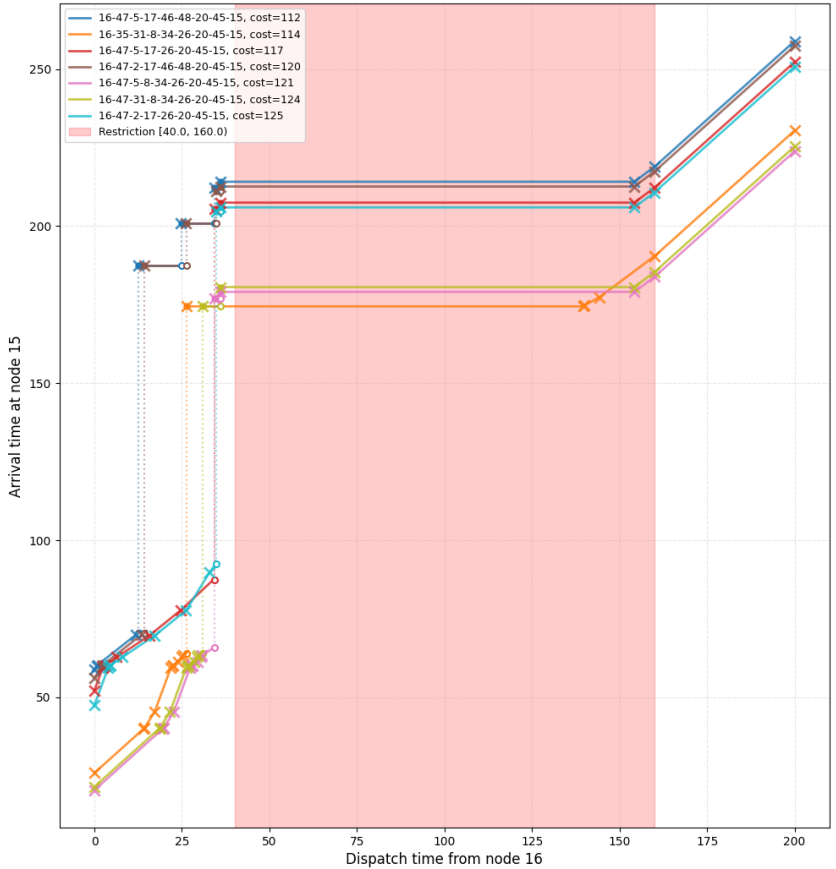


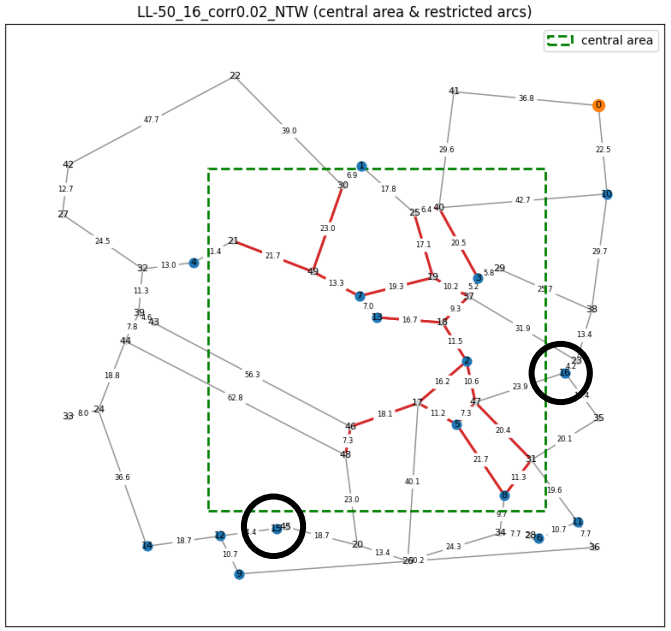
Road network (arc based) representation

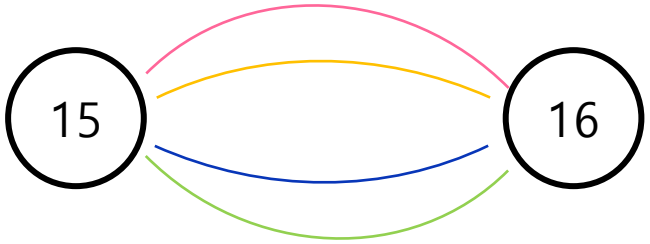
Appendix. Multigraph Construction

Constructing Pareto-Optimal Arcs in the Multigraph

- Select **non-dominated Pareto-optimal arcs(paths)** between node pairs based on **cost** and **arrival time**
- All candidate paths must be selected. At a single moment in time, SPRC is feasible. However, the result differs depending on the departure time—so how should this be handled in continuous time?

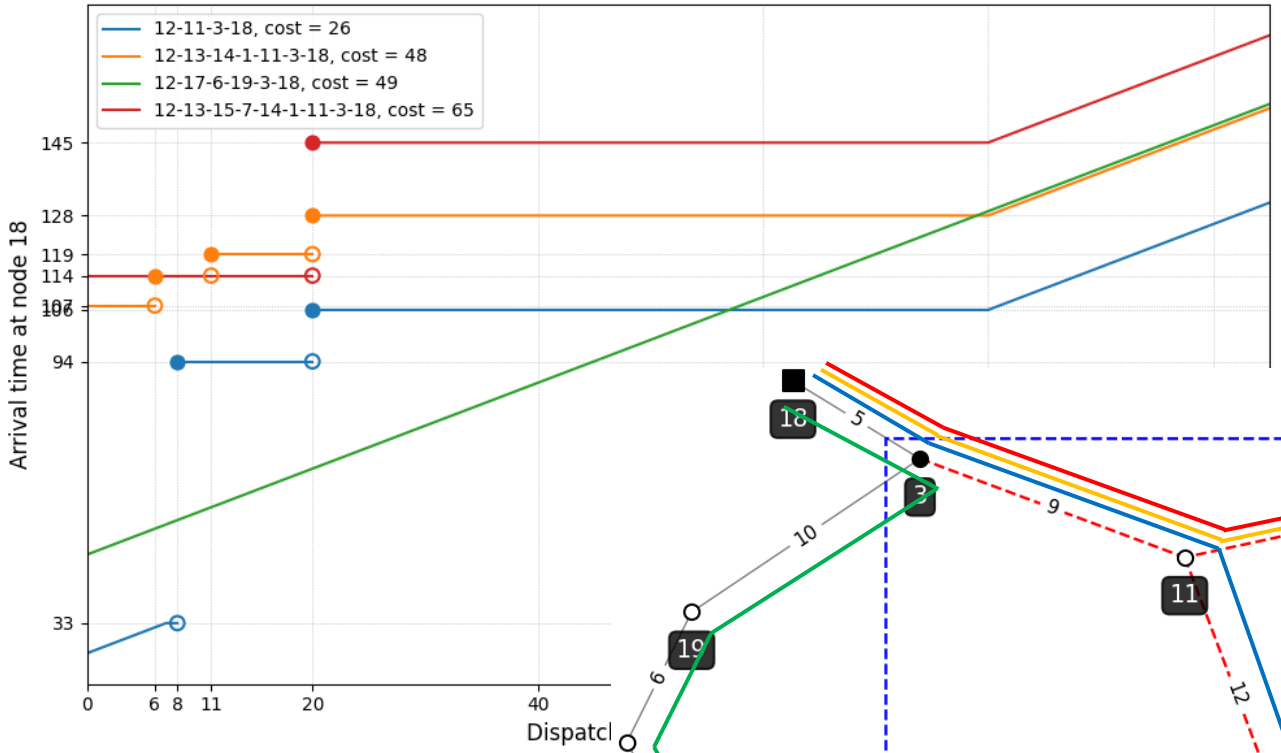




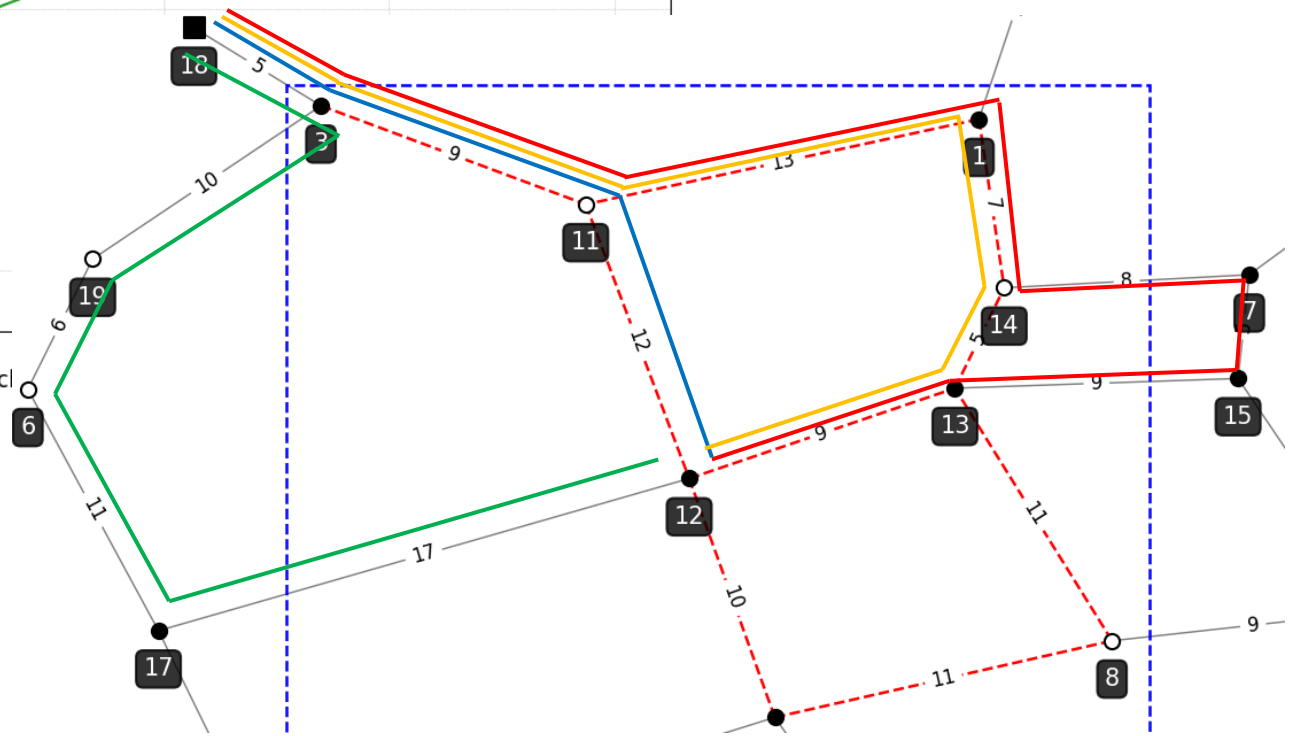


Appendix. Multigraph Construction

Constructing Pareto-Optimal Arcs in the Multigraph



(key paper's) →



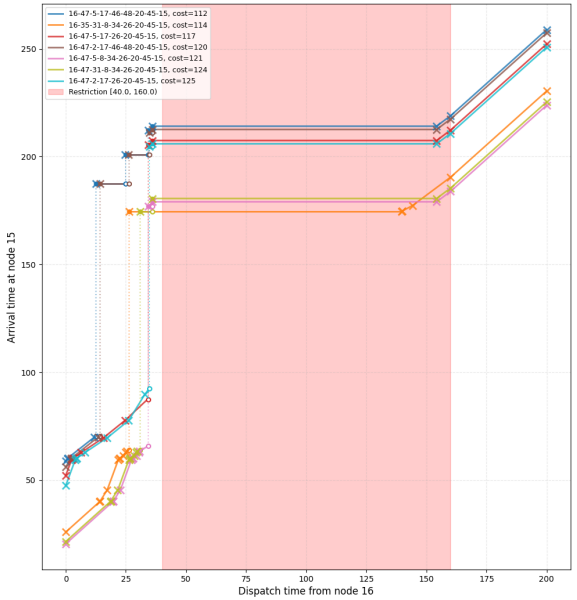
Algorithm 1 (Construction of the Multigraph)

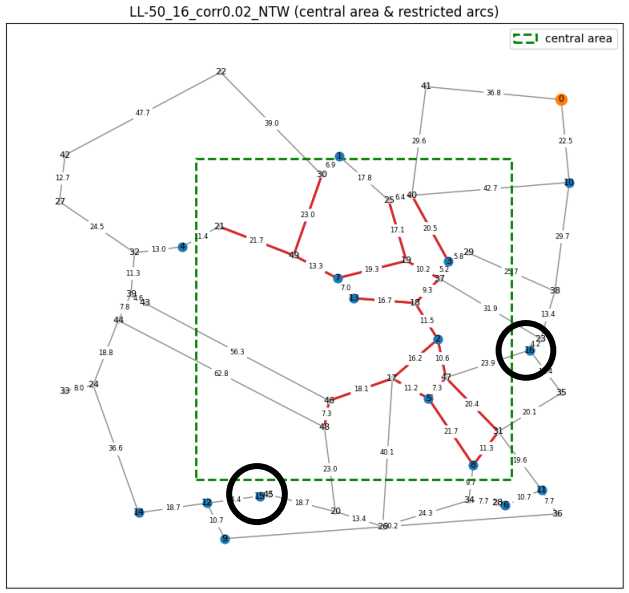
for each node pair $i_1 \in N'$ and $i_2 \in N'$ such that $q_{i_1} + q_{i_2} \leq Q$ **do**
  find all shortest paths between this pair
  for each shortest path $p = (a_0, a_1, a_2, \ldots, a_k)$ with $i_{a_0} = i_1$ and $j_{a_k} = i_2$ **do**
    if $\exists u \in \{0, \ldots, k\}$ such that $a_u \in A_R^p$ **then** mark p as time-dependent
    else mark p as time-independent and **break**
  if there exists a time-independent path **then** store this path as p^* and delete other min-cost paths
  else
    generate an artificial min-cost path p^* that has the earliest arrival time among all min-cost paths for each dispatch time and compute its breakpoints
    for $t_1 \in B_1^{p^*}$ **do**
      let t_2 be the breakpoint of δ^{p^*} after t_1
      $t \leftarrow t_1$
      while $t < t_2$ **do**
        use the labeling algorithm for the SPRC, where time is used as a resource and bounded by $\delta^{p^*}(t)$ and the vehicle leaves node i_1 at time t to find all Pareto-optimal paths
        if no path is found **then break**
        else
          let P' be the set of Pareto-optimal paths between i_1 and i_2 for dispatch time t, P^c contain all paths $p \in P'$ such that the arrival function δ^p is constant in the interval containing t and $P^i = P' \setminus P^c$
          $next \leftarrow t_2$
          for $p \in P'$ **do**
            let t' be the breakpoint of δ^p after t
            if $p \in P^c$ **then** p is Pareto-optimal within $[t, t')$
              $next \leftarrow \min\{next, t'\}$
            else
              if $\exists t'' \in [t, t')$ such that $\min_{p' \in P^c \cup \{p^*\}} \delta^{p'}(t) = \delta^p(t'')$ **then**
                p is Pareto-optimal within $[t, t'')$
              else p is Pareto-optimal within $[t, t')$
                $next \leftarrow \min\{next, t'\}$
          $t \leftarrow next$

Appendix. Multigraph Construction

Constructing Pareto-Optimal Arcs in the Multigraph (modified)

1. Unified Breakpoints: Uses breakpoints that integrate the time profile and restricted areas of the entire system.
2. SPRC (with slack): Dynamically sets resource boundaries by analyzing the slope of the artificial minimum-cost path.
3. Dynamic t2 Update: Dynamically updates sections through cross-analysis of the arrival times of candidate and reference paths.





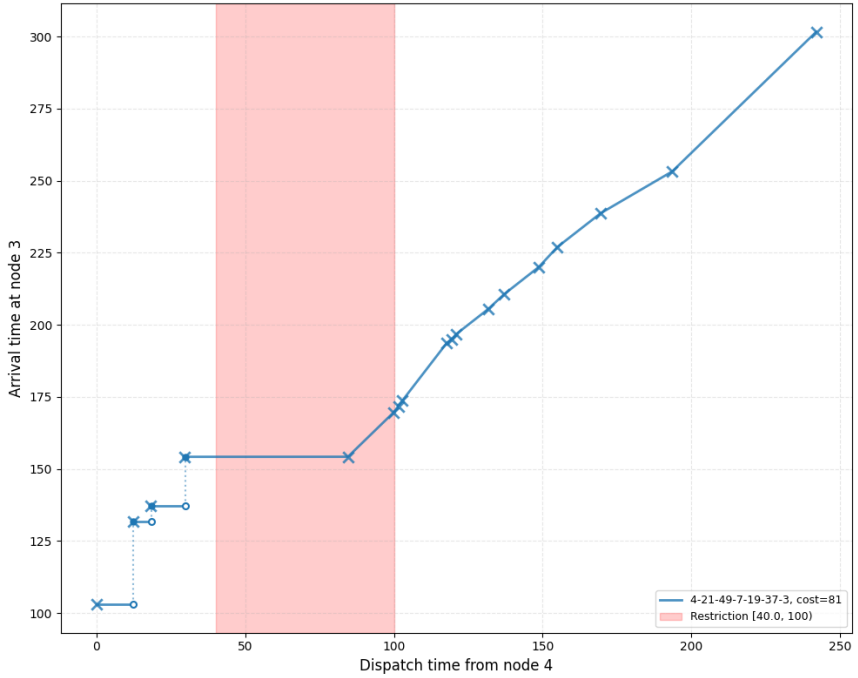
Appendix. Multigraph Construction

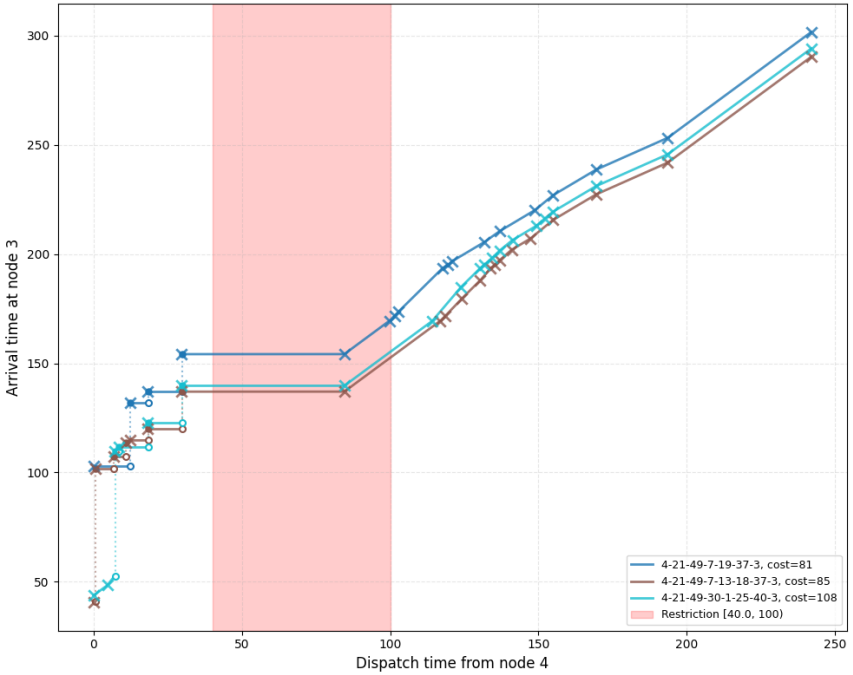
Constructing Pareto-Optimal Arcs in the Multigraph (modified)

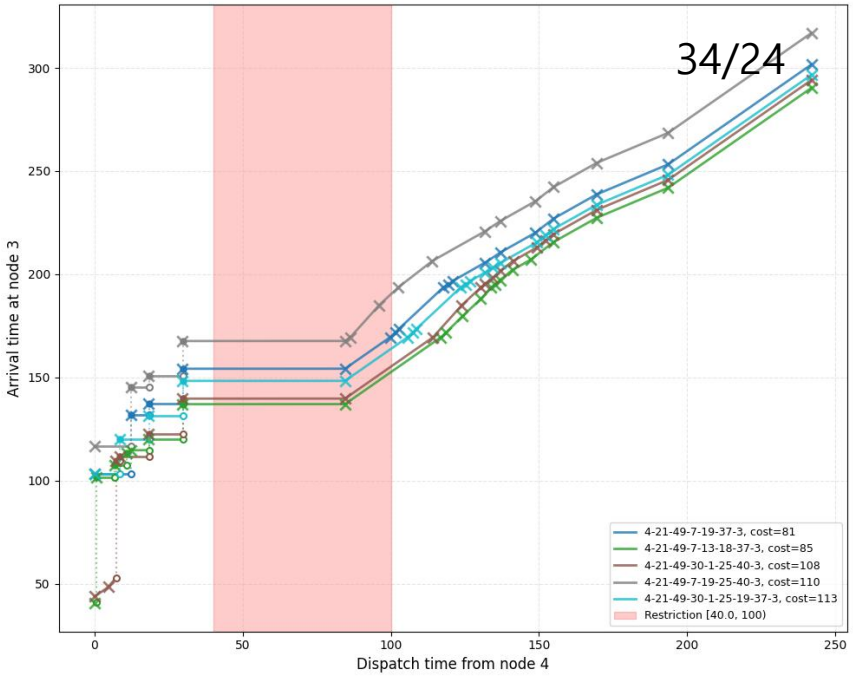
- Slope-change breakpoints
  - ✓ For which departure times does the arrival time at each arc along a path exactly coincide with the boundary of a speed-change period?

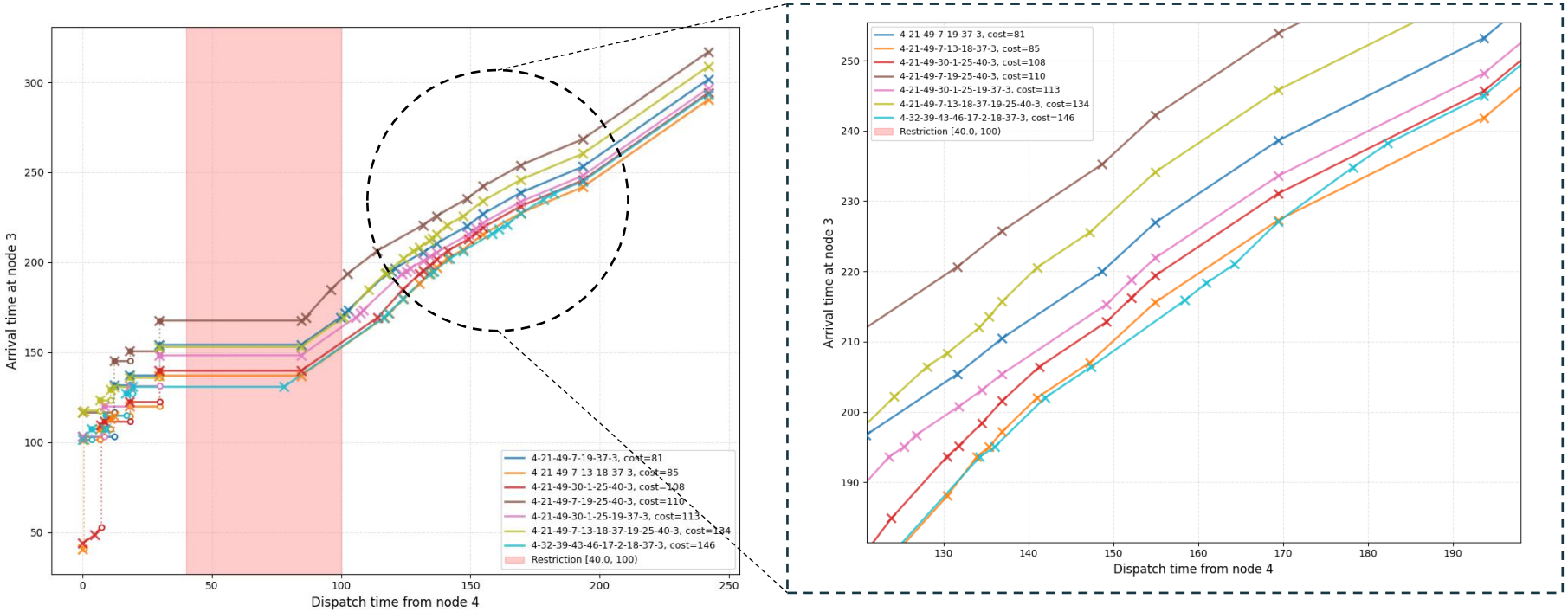
- Access-constraint breakpoints
  - ✓ For which departure times does the arrival time at each arc along a path exactly coincide with the start or end of an access-restriction period?

⇒ take the union of all breakpoints to generate candidate time points for exploration.







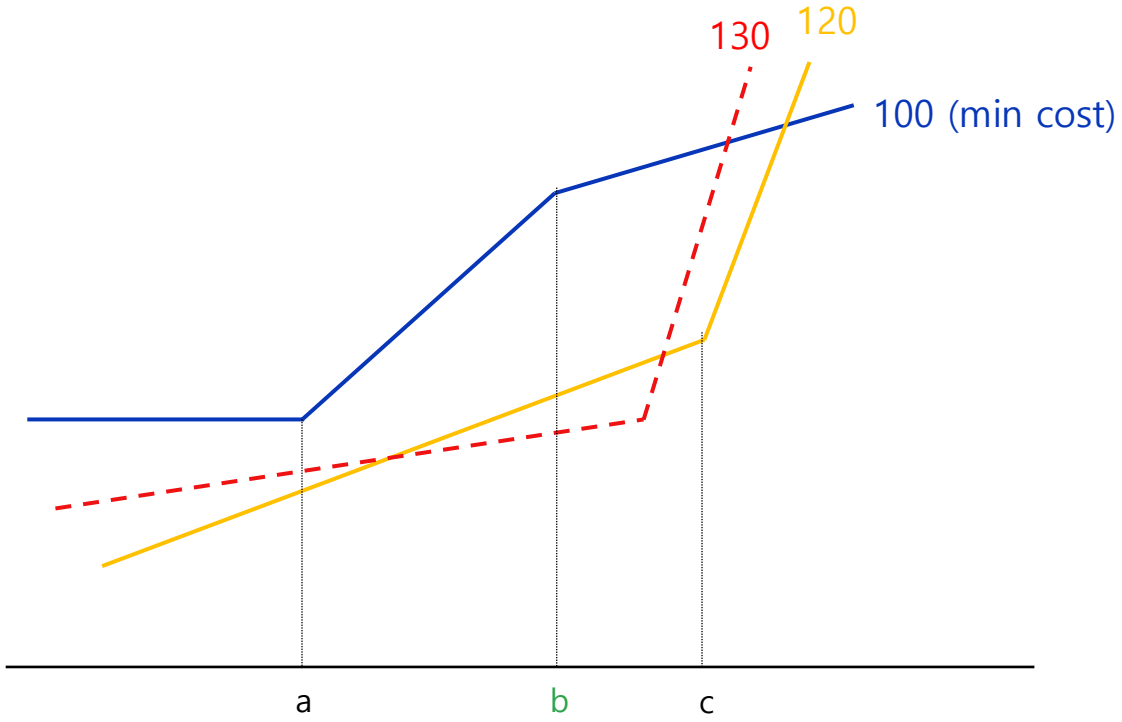


Continuously add the breakpoints of newly discovered paths to the set of candidate time points for exploration.

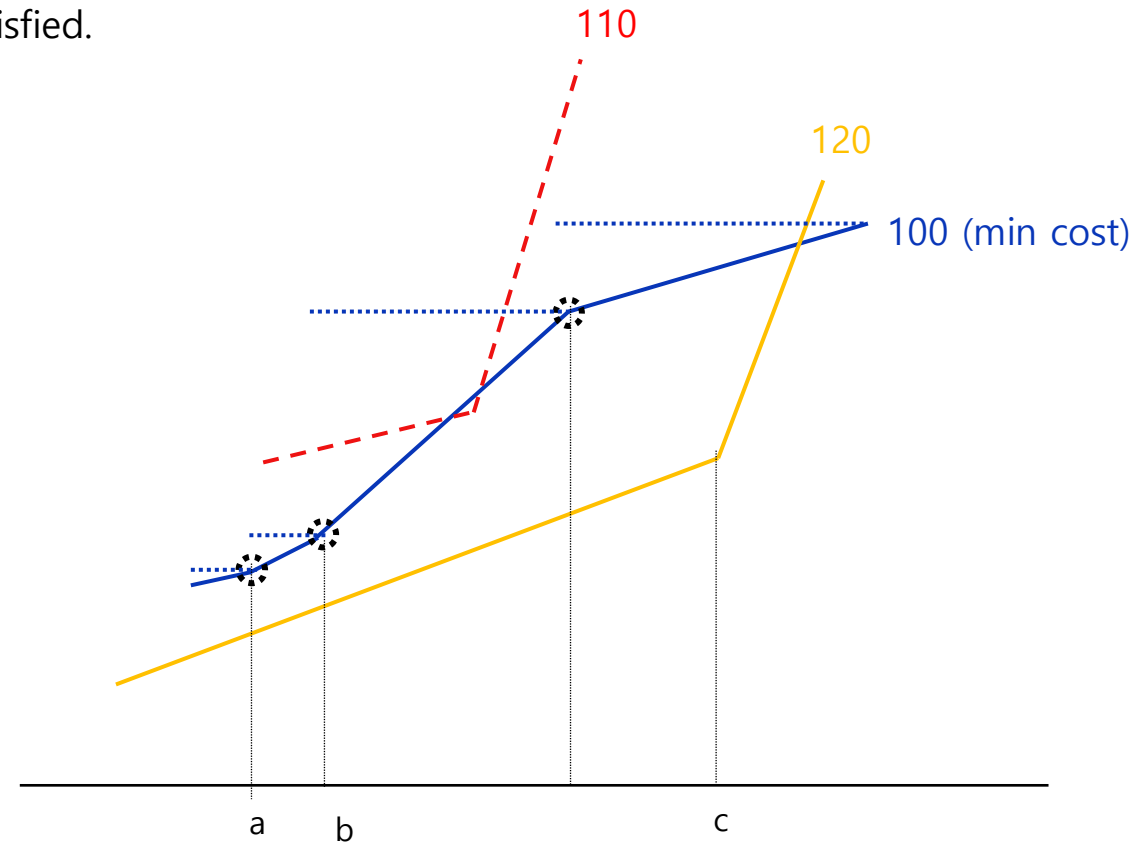
Appendix. Multigraph Construction

Constructing Pareto-Optimal Arcs in the Multigraph (modified)

- ✓ At each time point, running SPRC with a resource constraint based on the arrival time of the min-cost path may fail to find all Pareto-optimal paths.
- ✓ *To address this, grant a slack*
- ✓ Valid because the non-decreasing arrival (FIFO) property is satisfied.



Unable to find the red Pareto-optimal path



Able to find the red Pareto-optimal path

Appendix. Multigraph Construction

Constructing Pareto-Optimal Arcs in the Multigraph (modified)

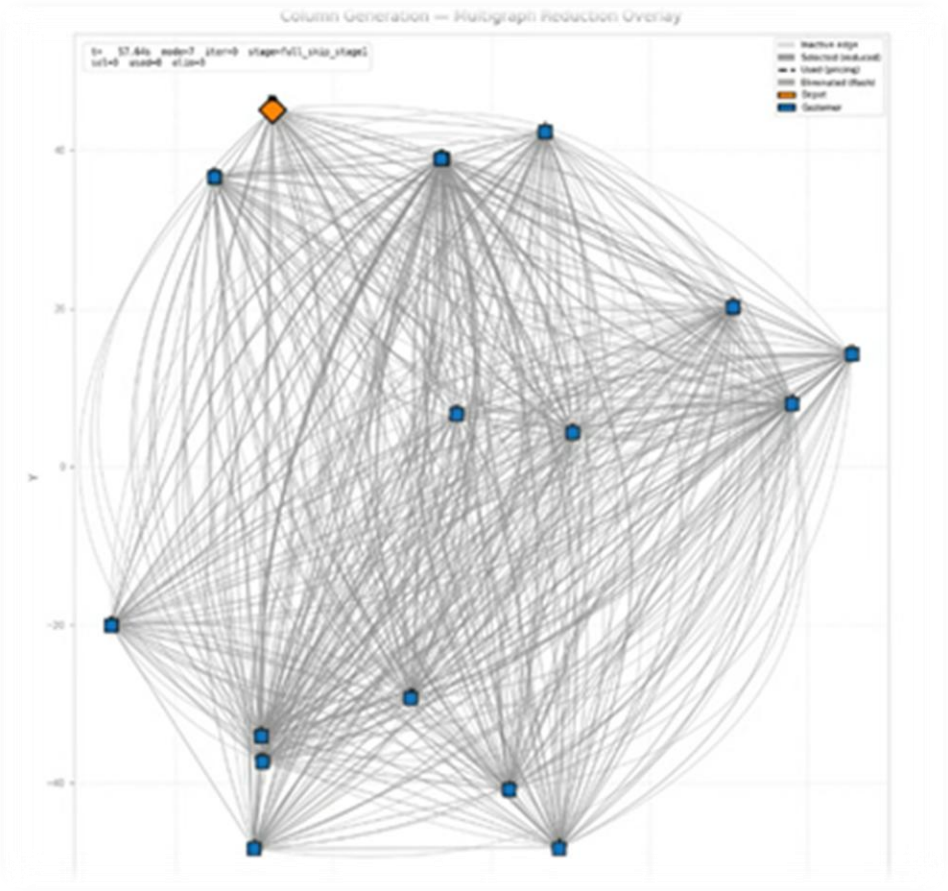
- ✓ Compared to the min-cost path,
- ✓ it may actually be dominated by another non–min-cost path.
- ✓ As a result, a wider (safer but less efficient) efficient period is obtained.

⇒ A sufficient condition for constructing a multigraph that contains the optimal solution, from both the path itself and the efficient-period perspective.

Appendix. Multigraph Construction

Multigraph Model for VRPSPD-AR

- Nodes
  - ✓ Inbound Cross Dock (single)
  - ✓ Suppliers
- Arcs
  - ✓ One Pareto-optimal "path"(arc) for every feasible pair respecting time-dependent access
  - ✓ Parallel arcs capture tradeoffs (cost vs. arrival time)



Appendix. Labeling Algorithm

- Pricing problem is solved using a forward labeling algorithm
  - ✓ Label L_v at node v is a tuple: (t_v, q_v, rc_v, S_v)
  - ✓ t_v : Earliest arrival time at v
  - ✓ q_v : Cumulative load delivered
  - ✓ rc_v : Cumulative reduced cost
  - ✓ S_v : Set of visited customers (for elementarity)

- Extension
  - ✓ Extending label at node v via edge $e = (v \rightarrow u)$
  - ✓ Feasible only if $q_j \leq Q, t_j \leq T, and\ j \notin S_i$

- Dominance Rule
  - ✓ Dominance rules are applied to prune suboptimal labels

Appendix. Dynamic τ Adjustment Logic

Algorithm 3 Dynamic τ Adjustment Logic

Require: Current τ_{curr}, Min/Max τ (τ_{min}, τ_{max}), Adjustment Factor δ, New Columns Count N_{col}

Ensure: Updated τ_{new}, Skip Flag for next Stage 1 $(Flag_{skip})$

```
if N_col < 5 then
    ▷ Too few columns found: Maintain τ, skip pricing next time
    τ_new ← τ_curr
    Flag_skip ← True
else if N_col > 20 then
    ▷ Too many columns: Lower τ (stricter pruning), do not skip
    τ_new ← max(τ_curr − δ, τ_min)
    Flag_skip ← False
else
    ▷ Moderate columns: Increase τ (relax pruning)
    τ_temp ← τ_curr + δ
    if τ_temp > τ_max then
        τ_new ← τ_max
        Flag_skip ← True        ▷ Clipped at max, safe to skip
    else
        τ_new ← τ_temp
        Flag_skip ← False
    end if
end if
return τ_new, Flag_skip
```

Appendix. Glossary

Table 1: Glossary of Symbols and Dimensions

Symbol	Definition	Dimension / Range
Sets & Indices		
$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	Multigraph with nodes $\mathcal{V}$ and edges $\mathcal{E}$.	$N = \lvert\mathcal{V}\rvert, E_{tot} = \lvert\mathcal{E}\rvert$
u, v	Indices for source and destination nodes.	$u, v \in \{1, \ldots, N\}$
k	Unique index for an edge connecting u to v.	$k \in \{1, \ldots, E_{tot}\}$
$\mathcal{P}$	Set of unique node pairs (u, v) connected by edges.	$\lvert\mathcal{P}\rvert \leq N^2$
p	Index for a specific node pair in $\mathcal{P}$.	$p \in \{1, \ldots, \lvert\mathcal{P}\rvert\}$
$\mathcal{E}_p$	Set of parallel edges belonging to pair p.	$\mathcal{E}_p \subseteq \mathcal{E}$
s, t	Indices for sequence segments (Travel Time, Efficient Period).	$s \in \{1 \ldots M_k\}$
h	Index for attention heads.	$h \in \{1, \ldots, H\}$
ℓ	Index for GNN layers.	$\ell \in \{1, \ldots, L_{GNN}\}$
Input Features		
$\mathbf{x}_u$	Raw feature vector for node u.	$\mathbb{R}^{D_{node}}$
$\mathbf{S}_k^{TT}$	Sequence of Travel Time segments for edge k.	$\mathbf{s} \in \mathbb{R}^{D_{seg}}$
$\mathbf{S}_k^{EP}$	Sequence of Efficient Periods for edge k.	$\mathbf{t} \in \mathbb{R}^{D_{per}}$
$\mathbf{e}_k^{scal}$	Scalar features for edge k (e.g., cost).	$\mathbb{R}^{D_{scal}}$
Learnable Parameters		
Θ_{enc}	Weights for sequence encoders (MLPs).	-
$\mathbf{W}^{GAT}$	Weights for Edge-Aware GAT layer.	$\mathbb{R}^{D_{out} \times D_{in}}$
$\mathbf{a}_{GAT}$	Attention vector for GAT.	$\mathbb{R}^{H \cdot D_{out}}$
Θ^{GCN}	Weights for Residual Gated GCN layers.	$\mathbb{R}^{D_{hid} \times D_{hid}}$
$\mathbf{W}_Q, \mathbf{W}_K$	Query/Key projections in Context Decoder.	$\mathbb{R}^{D_{head} \times D_{model}}$
$\mathbf{W}_{abs}$	Weights for Absolute Scoring branch.	$\mathbb{R}^{1 \times D_{model}}$
Hyperparameters		
C	Tanh clipping constant for the decoder.	Default: 10.0
H	Number of attention heads.	Default: 4

Appendix. GNN Architecture (1/4)

Raw Feature Encoding

- For each edge k, encode Travel Time (TT) and Efficient Periods (EP) using Masked Mean Pooling

$$\mathbf{z}_k^{TT} = \frac{1}{M_k} \sum_{s=1}^{M_k} \mathrm{ReLU}\left(\mathrm{MLP}_{TT}(\mathbf{s}_{k,s})\right) \in \mathbb{R}^{D_{TT}}$$

$$\mathbf{z}_k^{EP} = \frac{1}{L_k} \sum_{t=1}^{L_k} \mathrm{ReLU}\left(\mathrm{MLP}_{EP}(\mathbf{t}_{k,t})\right) \in \mathbb{R}^{D_{EP}}$$

- Output: Initial Edge Embedding

$$\mathbf{e}_k^{(0)} = \left[\mathbf{z}_k^{TT} \parallel \mathbf{z}_k^{EP} \parallel \mathbf{e}_k^{scal}\right] \in \mathbb{R}^{D_{edge}}$$

Appendix. GNN Architecture (2/4)

Graph Neural Network Backbone

- Node Projection

$$\mathbf{h}_u^{(0)} = \mathrm{ReLU}(\mathbf{W}_{node}\mathbf{x}_u) \in \mathbb{R}^{D_{hidden}}$$

- Layer 1: Edge-Aware Graph Attention Networks

$$\psi_{k,h} = \mathrm{LeakyReLU}\left(\mathbf{a}_h^T\left[\mathbf{W}_1\mathbf{h}_u^{(0)} + \mathbf{W}_2\mathbf{h}_v^{(0)} + \mathbf{W}_3\mathbf{e}_k^{(0)}\right]\right)$$

$$\alpha_{k,h} = \frac{\exp(\psi_{k,h})}{\sum_{j\in\mathcal{N}(u)}\exp(\psi_{(u,j),h})}$$

Compute attention coefficients per head

$$\mathbf{h}_u^{(1)} = \mathop{\Big\|}_{h=1}^{H}\left(\sum_{v\in\mathcal{N}(u)}\alpha_{(u,v),h}\mathbf{W}_5\mathbf{h}_v^{(0)}\right) + \mathbf{W}_4\mathbf{h}_u^{(0)}$$

Update node states by aggregating neighbor messages

Appendix. GNN Architecture (2/4)

Graph Neural Network Backbone

- Layers 2 to L_{GNN} : Residual Gated Graph Convolutional Networks

$$\eta_{uv} = \sigma\left(\Theta_3 \mathbf{h}_u^{(\ell)} + \Theta_4 \mathbf{h}_v^{(\ell)}\right)$$

Edge gating

$$\mathbf{h}_u^{(\ell+1)} = \mathbf{h}_u^{(\ell)} + \mathrm{ReLU}\left(\mathrm{BN}\left(\Theta_1 \mathbf{h}_u^{(\ell)} + \sum_{v \in \mathcal{N}(u)} \eta_{uv} \odot (\Theta_2 \mathbf{h}_v^{(\ell)})\right)\right)$$

Residual update for layers $l, \cdots, L_{GNN} - 1$

$$\mathbf{h}_u^{final} = \mathbf{h}_u^{(L_{GNN})}$$

Final node representation

- Edge Representation Construction

$$\mathbf{m}_k = \left[\mathbf{h}_u^{final} \parallel \mathbf{h}_v^{final} \parallel \mathbf{e}_k^{(0)}\right] \in \mathbb{R}^{D_{model}}$$

Unified representation for each edge k

Appendix. GNN Architecture (3/4)

Context-Aware Selection Decoder

- Pair Context Aggregation (Query): Mean embedding of all parallel edges

$$\mathbf{q}_p = \frac{1}{|\mathcal{E}_p|} \sum_{j \in \mathcal{E}_p} \mathbf{m}_j$$

- Relative Score: Edge k's prominence within its group

$$\text{Score}_{rel}(k) = \frac{1}{H} \sum_{h=1}^{H} \frac{(\mathbf{W}_{Q,h}\mathbf{q}_p)^T (\mathbf{W}_{K,h}\mathbf{m}_k)}{\sqrt{D_{head}}}$$

- Absolute Score: Intrinsic quality of edge k

$$\text{Score}_{abs}(k) = \mathbf{W}_{abs}\mathbf{m}_k + b_{abs}$$

Appendix. GNN Architecture (4/4)

Final Output and Loss

- Final Logit

$$\hat{y}_k = C \cdot \tanh\left(\mathrm{Score}_{rel}(k) + \mathrm{Score}_{abs}(k)\right)$$

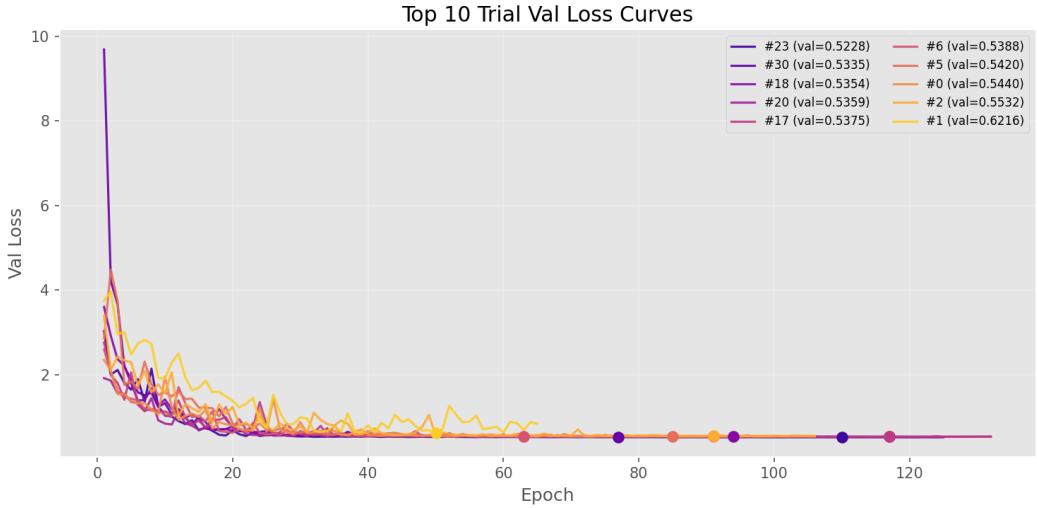
- Loss Function: Binary Cross Entropy Loss

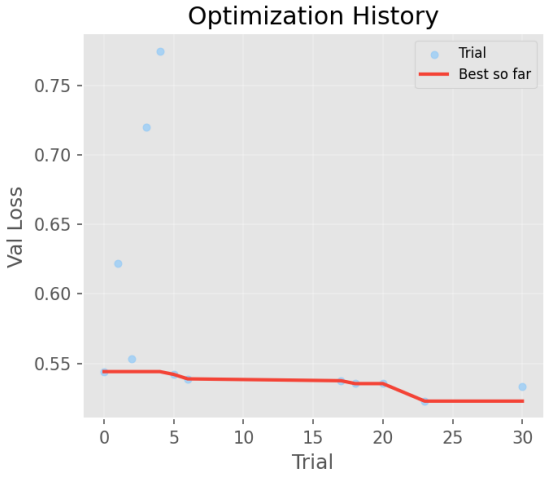
$$\mathcal{L} = -\frac{1}{E_{tot}} \sum_{k=1}^{E_{tot}} \left[y_k \cdot \log(\sigma(\hat{y}_k)) + (1 - y_k) \cdot \log(1 - \sigma(\hat{y}_k)) \right]$$

Appendix. Hyperparameter Tuning

Final Output and Loss

- Learning History





- Tuning Detail

Hyperparameter	Type	Search Range / Choices	Selected Value
num_attention_heads	Categorical	$\{2, 4, 8\}$	4
hidden_dim	Categorical	$\{64, 128, 256\}$	64
num_gated_layers	Integer	$[2, 6]$	2
dropout	Float	$[0.1, 0.5]$ (Step: 0.05)	0.30
learning_rate	Log-Uniform	$[1 \times 10^{-4}, 5 \times 10^{-3}]$	2.66×10^{-3}
weight_decay	Log-Uniform	$[1 \times 10^{-6}, 1 \times 10^{-3}]$	2.42×10^{-4}
alpha_l1	Log-Uniform	$[1 \times 10^{-8}, 1 \times 10^{-5}]$	3.29×10^{-8}